MySQL

Complete Mastery Guide

PART 1 — Fundamentals

Data Types · CREATE · INSERT · SELECT · WHERE · ORDER BY · NULL · UPDATE · DELETE · LIMIT · Operators · LIKE · IN · BETWEEN · GROUP BY · HAVING · JOINS · UNION · CASE · Constraints

★ Beginner Friendly ★ Before & After Tables ★ Interview Q&A ★ Multiple Examples ★

Part 1 — Table of Contents

1. Introduction to MySQL & RDBMS Concepts
2. MySQL Data Types — Numeric, String, Date, Other
3. CREATE DATABASE & USE
4. CREATE TABLE — Syntax, Constraints, Foreign Keys
5. INSERT INTO — Single, Multi-Row, ON DUPLICATE KEY, INSERT SELECT
6. SELECT — Basic Queries, Aliases, Expressions, DISTINCT
7. WHERE Clause — Filters, Comparisons, Date Filters
8. AND / OR / NOT — Logical Operators
9. ORDER BY — Single, Multiple Columns, Custom Sort
10. NULL Values — IS NULL, IS NOT NULL, IFNULL, COALESCE, NULLIF
11. UPDATE — Single Row, Multi-Row, UPDATE with JOIN
12. DELETE — DELETE with WHERE, DELETE with JOIN, vs TRUNCATE
13. LIMIT & OFFSET — Top-N, Pagination
14. MIN and MAX Functions
15. COUNT, AVG, SUM — Aggregates
16. LIKE Operator & Wildcards (%, _)
17. IN Operator — Value List & Subquery
18. BETWEEN Operator
19. Aliases (AS) — Column and Table Aliases
20. MySQL Operators — Full Reference
21. INNER JOIN
22. LEFT JOIN
23. RIGHT JOIN
24. CROSS JOIN
25. SELF JOIN
26. UNION & UNION ALL
27. GROUP BY
28. HAVING
29. CASE Expression
30. EXISTS Operator
31. ANY and ALL Operators
32. Constraints — CHECK, UNIQUE, NOT NULL, DEFAULT
33. ALTER TABLE — Full Reference
34. DROP TABLE & TRUNCATE TABLE
35. AUTO_INCREMENT
36. MySQL Comments
37. Part 1 Interview Questions & Answers

1. Introduction to MySQL & RDBMS

⍰ Interview-Ready Definition

MySQL is the world's most popular open-source Relational Database Management System (RDBMS). It stores data in structured TABLES (rows & columns), uses SQL (Structured Query Language) to read/write data, and supports ACID transactions, foreign keys, stored procedures, triggers, and views. Developed in 1995 and now owned by Oracle Corporation.

Key Terminology

Term	Definition	EduShop Example
Database	Container of related tables	edushop
Table	Grid of rows and columns	students, courses, orders
Row / Record	One complete data entry	One student's full details
Column / Field	One attribute of data	full_name, email, price
Primary Key (PK)	Uniquely identifies each row; NOT NULL	student_id = 1
Foreign Key (FK)	References PK in another table	order.student_id → students.student_id
Index	Fast-lookup data structure (B-Tree)	INDEX on email column
Schema	Structure definition of all tables	CREATE TABLE scripts
Query	SQL command to read/write data	SELECT * FROM students
Transaction	Atomic unit — all succeed or all fail	INSERT + UPDATE done together
NULL	Unknown/missing value — not zero or empty string	phone = NULL (not provided)

RDBMS vs DBMS vs NoSQL

Feature	DBMS (File)	RDBMS — MySQL	NoSQL — MongoDB
Data storage	Flat files	Related tables	Documents/Key-Value
Table relationships	No	Yes (Foreign Keys)	No
SQL language	No	Yes (full SQL)	No (own query lang)
ACID properties	No	Yes (InnoDB)	Partial
Best for	Simple data	Structured relational data	Unstructured/huge scale

? What is the difference between SQL and MySQL?

✅SQL (Structured Query Language) is a STANDARD LANGUAGE for querying and managing databases — it is not software. MySQL is SOFTWARE (an RDBMS engine) that IMPLEMENTS the SQL standard. Think of SQL as a language like English, and MySQL as a person who speaks it. PostgreSQL, Oracle, and SQL Server also use SQL.

? What are the five sub-languages of SQL?

✅DDL (Data Definition Language) — CREATE, ALTER, DROP, TRUNCATE. DML (Data Manipulation Language) — INSERT, UPDATE, DELETE. DQL (Data Query Language) — SELECT. DCL (Data Control Language) — GRANT, REVOKE. TCL (Transaction Control Language) — COMMIT, ROLLBACK, SAVEPOINT.

❓ What is the default storage engine in MySQL and why does it matter?

✅InnoDB is the default storage engine since MySQL 5.5. It supports ACID transactions, row-level locking, foreign keys, and crash recovery. The older MyISAM engine does NOT support transactions or foreign keys — always use InnoDB for production applications.

EduShop — Sample Database (Used in All Examples)

Every example throughout Part 1 and Part 2 uses the EduShop database — an online course platform where students buy, take, and review courses. Load this script once and use it everywhere.

Table Overview

Table	Columns	Purpose
categories	category_id, category_name	Course categories (Data Science, Web Dev...)
instructors	instructor_id, full_name, email, city, rating, joined_date	Course instructors
courses	course_id, title, category_id, instructor_id, price, duration_hrs, level, published_at	Available courses
students	student_id, full_name, email, city, phone, joined_date, is_active	Platform students
orders	order_id, student_id, course_id, order_date, amount_paid, coupon_code, status	Purchase records
reviews	review_id, student_id, course_id, rating, comment, reviewed_at	Student reviews

Full Schema + Data Script

```
-- ═══════════════════════════════════════════════════
-- EduShop Database — Full Schema + Sample Data Script
-- Run this once; every example in Parts 1 & 2 uses it
-- ═══════════════════════════════════════════════════
CREATE DATABASE IF NOT EXISTS edushop;
USE edushop;

-- ── CATEGORIES ──────────────────────────────────
CREATE TABLE categories (
  category_id   INT PRIMARY KEY AUTO_INCREMENT,
  category_name VARCHAR(100) NOT NULL,
  created_at    DATETIME DEFAULT CURRENT_TIMESTAMP
);

-- ── INSTRUCTORS ─────────────────────────────────
CREATE TABLE instructors (
  instructor_id INT PRIMARY KEY AUTO_INCREMENT,
  full_name     VARCHAR(150) NOT NULL,
  email         VARCHAR(150) UNIQUE NOT NULL,
  city          VARCHAR(80),
  rating        DECIMAL(3,2),
  joined_date   DATE
);

-- ── COURSES ─────────────────────────────────────
CREATE TABLE courses (
  course_id     INT PRIMARY KEY AUTO_INCREMENT,
  title         VARCHAR(200) NOT NULL,
  category_id   INT,
```

```sql
  instructor_id INT,
  price         DECIMAL(8,2) NOT NULL DEFAULT 0.00,
  duration_hrs  INT,
  level         ENUM('Beginner','Intermediate','Advanced'),
  published_at  DATE,
  is_active     TINYINT(1) DEFAULT 1,
  FOREIGN KEY (category_id)
      REFERENCES categories(category_id) ON DELETE SET NULL ON UPDATE CASCADE,
  FOREIGN KEY (instructor_id)
      REFERENCES instructors(instructor_id) ON DELETE RESTRICT ON UPDATE CASCADE
);

-- ── STUDENTS ──────────────────────────────────────────
CREATE TABLE students (
  student_id  INT PRIMARY KEY AUTO_INCREMENT,
  full_name   VARCHAR(150) NOT NULL,
  email       VARCHAR(150) UNIQUE NOT NULL,
  city        VARCHAR(80),
  phone       VARCHAR(20),
  joined_date DATE,
  is_active   TINYINT(1) DEFAULT 1
);

-- ── ORDERS ────────────────────────────────────────────
CREATE TABLE orders (
  order_id    INT PRIMARY KEY AUTO_INCREMENT,
  student_id  INT NOT NULL,
  course_id   INT NOT NULL,
  order_date  DATETIME DEFAULT CURRENT_TIMESTAMP,
  amount_paid DECIMAL(8,2),
  coupon_code VARCHAR(20),
  status      ENUM('pending','completed','refunded') DEFAULT 'pending',
  FOREIGN KEY (student_id) REFERENCES students(student_id) ON DELETE RESTRICT,
  FOREIGN KEY (course_id)  REFERENCES courses(course_id)  ON DELETE RESTRICT
);

-- ── REVIEWS ───────────────────────────────────────────
CREATE TABLE reviews (
  review_id   INT PRIMARY KEY AUTO_INCREMENT,
  student_id  INT,
  course_id   INT,
  rating      TINYINT CHECK (rating BETWEEN 1 AND 5),
  comment     TEXT,
  reviewed_at DATETIME DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (student_id) REFERENCES students(student_id),
  FOREIGN KEY (course_id)  REFERENCES courses(course_id)
);

-- ── INSERT DATA ───────────────────────────────────────
INSERT INTO categories (category_name) VALUES
  ('Data Science'),('Web Development'),('Design'),
  ('Business'),('Mobile Development');

INSERT INTO instructors (full_name,email,city,rating,joined_date) VALUES
  ('Ravi Kumar',   'ravi@edushop.com',   'Hyderabad', 4.80, '2020-01-15'),
  ('Priya Sharma', 'priya@edushop.com',  'Bangalore',  4.60, '2020-03-20'),
  ('Arjun Mehta',  'arjun@edushop.com',  'Mumbai',     4.90, '2019-11-05'),
  ('Sneha Reddy',  'sneha@edushop.com',  'Chennai',    4.30, '2021-07-10'),
  ('Vikram Singh', 'vikram@edushop.com', 'Delhi',      NULL, '2022-02-18');

INSERT INTO courses
(title,category_id,instructor_id,price,duration_hrs,level,published_at) VALUES
  ('Python for Data Science',    1,1,1499.00,40,'Beginner',    '2021-01-10'),
  ('Machine Learning A-Z',       1,3,2999.00,60,'Advanced',    '2021-06-15'),
  ('React JS Complete Guide',    2,2,1999.00,35,'Intermediate','2022-03-01'),
```

```
  ('UI/UX Design Fundamentals',  3,4, 999.00,20,'Beginner',    '2022-08-20'),
  ('MySQL Mastery',              2,1,1299.00,25,'Intermediate','2023-01-05'),
  ('Flutter Mobile Dev',         5,3,2499.00,50,'Advanced',    '2022-11-12'),
  ('Business Analytics',         4,2,1799.00,30,'Intermediate','2021-09-30'),
  ('Advanced React Patterns',    2,2,2299.00,28,'Advanced',    '2023-04-18'),
  ('Excel for Business',         4,4, 699.00,15,'Beginner',    '2020-05-01'),
  ('Deep Learning',              1,3,3499.00,70,'Advanced',    '2023-07-01');

INSERT INTO students (full_name,email,city,phone,joined_date) VALUES
  ('Amit Joshi',   'amit@gmail.com',   'Hyderabad','9000000001','2022-01-10'),
  ('Meera Nair',   'meera@gmail.com',  'Kochi',    '9000000002','2022-03-15'),
  ('Rahul Gupta',  'rahul@gmail.com',  'Delhi',    '9000000003','2021-11-20'),
  ('Anjali Singh', 'anjali@gmail.com', 'Mumbai',   '9000000004','2023-02-05'),
  ('Sanjay Patel', 'sanjay@gmail.com', 'Ahmedabad','9000000005','2022-07-18'),
  ('Kavya Reddy',  'kavya@gmail.com',  'Hyderabad','9000000006','2023-01-01'),
  ('Deepak Verma', 'deepak@gmail.com', 'Pune',     NULL,        '2021-08-25'),
  ('Nisha Agarwal','nisha@gmail.com',  'Jaipur',   '9000000008','2022-09-30'),
  ('Rohit Sharma', 'rohit@gmail.com',  'Bangalore','9000000009','2020-12-01'),
  ('Pooja Iyer',   'pooja@gmail.com',  'Chennai',  '9000000010','2023-05-10');

INSERT INTO orders
  (student_id,course_id,order_date,amount_paid,coupon_code,status) VALUES
  (1,1, '2022-02-10 10:00:00',1499.00,NULL,     'completed'),
  (1,3, '2022-04-05 11:30:00',1799.00,'SAVE10', 'completed'),
  (2,2, '2022-05-01 09:00:00',2999.00,NULL,     'completed'),
  (2,5, '2023-02-14 15:00:00',1299.00,'SAVE10', 'completed'),
  (3,1, '2021-12-20 14:00:00',1499.00,NULL,     'completed'),
  (3,4, '2022-01-05 08:00:00', 999.00,NULL,     'refunded'),
  (4,6, '2023-03-10 10:00:00',2499.00,'FLAT200','completed'),
  (5,7, '2022-08-12 16:00:00',1799.00,NULL,     'completed'),
  (5,9, '2022-09-01 09:30:00', 699.00,NULL,     'completed'),
  (6,1, '2023-01-15 11:00:00',1499.00,NULL,     'completed'),
  (6,2, '2023-02-20 13:00:00',2999.00,'SAVE10', 'completed'),
  (7,3, '2021-09-10 10:00:00',1999.00,NULL,     'completed'),
  (8,5, '2022-10-05 14:00:00',1299.00,NULL,     'pending'),
  (9,10,'2021-01-15 09:00:00',3499.00,NULL,     'completed'),
  (10,8,'2023-06-01 12:00:00',2299.00,NULL,     'completed');

INSERT INTO reviews (student_id,course_id,rating,comment) VALUES
  (1,1,5,'Excellent! Loved the practical examples.'),
  (1,3,4,'Good content, could use more exercises.'),
  (2,2,5,'Best ML course on the platform!'),
  (3,1,4,'Very helpful for beginners.'),
  (5,7,3,'Average content, expected more depth.'),
  (6,1,5,'Perfect for starters.'),
  (6,2,5,'Instructor is amazing!'),
  (9,10,4,'Challenging but worth it.'),
  (10,8,5,'Loved the advanced patterns section.');
```

2. MySQL Data Types

Interview-Ready Definition

A data type defines what kind of value a column can store. Choosing the correct data type ensures data integrity, saves storage space, and improves query performance. MySQL groups types into: Numeric, String/Text, Date & Time, and Spatial/JSON.

Numeric Types

Type	Storage	Range / Notes	Best Use
TINYINT	1 byte	-128 to 127 (UNSIGNED: 0–255)	Flags (0/1), small status codes
SMALLINT	2 bytes	-32,768 to 32,767	Age, small counters
MEDIUMINT	3 bytes	-8M to 8M	Medium-range IDs
INT	4 bytes	-2.1B to 2.1B	Primary keys, quantities
BIGINT	8 bytes	Very large integers	Large IDs, timestamps
DECIMAL(p,s)	Varies	Exact p digits, s decimal places	MONEY — always use this
FLOAT	4 bytes	~7 significant digits (approx)	Scientific data only
DOUBLE	8 bytes	~15 significant digits (approx)	High-precision floats
BOOL/BOOLEAN	1 byte	0 = false, 1 = true	Boolean flags

String / Text Types

Type	Max Size	Notes	Best Use
CHAR(n)	255 chars	Fixed length — always uses n bytes; pads with spaces	Country codes (IN, US), fixed codes
VARCHAR(n)	65,535 bytes	Variable length — uses actual bytes + 1–2 overhead	Names, emails, titles, URLs
TINYTEXT	255 bytes	No DEFAULT value allowed	Very short text
TEXT	65,535 bytes	No DEFAULT; cannot be fully indexed	Comments, descriptions, articles
MEDIUMTEXT	16 MB	—	Large documents
LONGTEXT	4 GB	—	Huge text blobs
ENUM(vals)	1–2 bytes	Exactly one value from a predefined list	Status fields, levels, types
SET(vals)	1–8 bytes	Zero or more values from a predefined list	Tags, multi-select options
JSON	Varies	Valid JSON document; MySQL 5.7+	Semi-structured/flexible data

Date & Time Types

Type	Format	Range	Key Difference / Use
DATE	YYYY-MM-DD	1000-01-01 to 9999-12-31	Date only — no time component. Birthdates, publish dates
TIME	HH:MM:SS	-838:59:59 to 838:59:59	Time only. Durations, time-of-day values
DATETIME	YYYY-MM-DD HH:MM:SS	1000 to 9999	Stores as-is — no time zone conversion. Historical records
TIMESTAMP	YYYY-MM-DD HH:MM:SS	1970 to 2038	Converts to UTC on save, back on read. Use for created_at/updated_at
YEAR	YYYY	1901 to 2155	Year only. Manufacturing year, vintage

Example 1: Why Data Type Choice Matters — Practical Examples

```
-- ✅CORRECT: Use DECIMAL for money (exact)
CREATE TABLE orders (
  amount_paid DECIMAL(10,2)   -- Exact: 1234.56
);

-- ❌WRONG: FLOAT for money (approximate — rounding errors!)
CREATE TABLE orders_bad (
  amount_paid FLOAT           -- May store 1499.99999... instead of 1500.00
);

-- ✅CORRECT: VARCHAR for variable-length names
full_name VARCHAR(150)         -- Uses only needed bytes

-- ❌WRONG: CHAR for names (wastes space)
full_name CHAR(150)            -- Always uses 150 bytes even for 'Ami'

-- ✅CORRECT: TIMESTAMP for auto-recording create time
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP

-- ✅CORRECT: ENUM for fixed options
level ENUM('Beginner','Intermediate','Advanced')
-- Stored as 1 byte; enforced by DB; shows readable values
```

How it works: Each column type choice has real consequences: DECIMAL vs FLOAT = exact vs approximate values. CHAR vs VARCHAR = wasted space vs efficient storage. TIMESTAMP vs DATETIME = timezone-aware vs stored as-is.

? Why use DECIMAL instead of FLOAT for money?

✅FLOAT uses binary fractions internally — 0.1 + 0.2 can equal 0.30000000000000004. DECIMAL stores EXACT decimal digits. For financial calculations ALWAYS use DECIMAL(10,2) to avoid rounding errors that compound over millions of transactions.

? What is the difference between CHAR and VARCHAR?

✅CHAR(n) always uses exactly n bytes — pads with spaces if shorter. VARCHAR(n) uses only the actual string length + 1 or 2 bytes overhead. Use CHAR for fixed-length codes (country codes: IN, US). Use VARCHAR for variable text like names, emails.

❓ What is the difference between DATETIME and TIMESTAMP?

✅DATETIME stores the date-time as-is with NO timezone conversion — what you put in is what you get back. TIMESTAMP converts to UTC for storage and converts back based on the session timezone on retrieval. TIMESTAMP range is limited to 2038. Use DATETIME for historical records; TIMESTAMP for auto-updated columns like created_at.

3. CREATE DATABASE & USE

Interview-Ready Definition

CREATE DATABASE creates a new empty database container. USE selects it as the active database for all subsequent statements. SHOW DATABASES lists all databases on the server. DROP DATABASE permanently removes it along with all its tables — there is NO undo.

Example 1: Create and Select a Database

```
-- Create a new database
CREATE DATABASE edushop;

-- Safe version (no error if already exists)
CREATE DATABASE IF NOT EXISTS edushop;

-- Select the database to use
USE edushop;

-- Show all databases on the server
SHOW DATABASES;

-- Show currently selected database
SELECT DATABASE();
```

Output:

```
+--------------------+
| Database           |
+--------------------+
| edushop            |
| information_schema |
| mysql              |
| performance_schema |
+--------------------+

SELECT DATABASE() → edushop
```

How it works: IF NOT EXISTS prevents an error if the database already exists — essential for scripts that run multiple times (migrations, setups).

Pitfall: DROP DATABASE permanently deletes ALL tables, data, views, stored procedures inside it. There is no Recycle Bin. Always backup before dropping.

Example 2: Drop and Recreate — Test Environment Setup

```
-- Drop old test database safely
DROP DATABASE IF EXISTS test_db;

-- Create fresh
CREATE DATABASE test_db
  CHARACTER SET utf8mb4       -- Support all Unicode characters (emojis included)
  COLLATE utf8mb4_unicode_ci; -- Case-insensitive Unicode comparison

-- Switch to it
USE test_db;

-- Verify
SELECT DATABASE();   -- test_db
```

How it works: CHARACTER SET utf8mb4 supports full Unicode including emoji. COLLATE determines sort order and case sensitivity. utf8mb4_unicode_ci is the recommended choice for most applications.

? How do you check which database is currently selected?

✅SELECT DATABASE(); — returns the name of the active database, or NULL if none is selected.

? What is the difference between a schema and a database in MySQL?

✅In MySQL, DATABASE and SCHEMA are synonyms — CREATE DATABASE and CREATE SCHEMA do exactly the same thing. In other RDBMS like Oracle or SQL Server, a schema is a namespace within a database, but MySQL treats them as equivalent.

4. CREATE TABLE — Syntax, Constraints, Foreign Keys

Interview-Ready Definition

CREATE TABLE defines the structure of a new table — column names, data types, and constraints. Constraints are rules enforced by the DATABASE ENGINE (not application code) to maintain data integrity. Types: PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL, CHECK, DEFAULT.

All Constraint Types at a Glance

Constraint	Purpose	NULL Allowed?	Example
PRIMARY KEY	Unique row identifier — one per table	No	student_id INT PRIMARY KEY
FOREIGN KEY	Links to PK in another table	Yes	REFERENCES students(student_id)
UNIQUE	No duplicate values in column(s)	Yes (one NULL allowed)	UNIQUE (email)
NOT NULL	Column value is mandatory	No	full_name VARCHAR(150) NOT NULL
CHECK	Value must satisfy a condition	Yes	CHECK (price >= 0)
DEFAULT	Fallback value when column is omitted	—	DEFAULT CURRENT_TIMESTAMP
AUTO_INCREMENT	Auto-generate next integer on insert	No	INT AUTO_INCREMENT

Example 1: Simple CREATE TABLE

▶ Table Before — No Structure Yet

(Table does not exist yet — we are about to create it)

```
CREATE TABLE categories (
  category_id   INT           NOT NULL AUTO_INCREMENT,
  category_name VARCHAR(100)  NOT NULL,
  description   TEXT,
  created_at    DATETIME      DEFAULT CURRENT_TIMESTAMP,
  PRIMARY KEY (category_id)
);
```

Output:

```
Query OK, 0 rows affected
```

▶ Table After CREATE — Structure (no rows yet)

Column	Type	Constraints	Default
category_id	INT	PRIMARY KEY, NOT NULL, AUTO_INCREMENT	(auto)
category_name	VARCHAR(100)	NOT NULL	(required)

Column	Type	Constraints	Default
description	TEXT	NULL allowed	NULL
created_at	DATETIME	NULL allowed	CURRENT_TIMESTAMP

How it works: AUTO_INCREMENT makes category_id fill automatically: first row gets 1, next gets 2, etc. NOT NULL on category_name means every INSERT must include a name.

Example 2: CREATE TABLE with FOREIGN KEY and ON DELETE / ON UPDATE Actions

▶ **Parent tables must exist first (categories, instructors)**

```
CREATE TABLE courses (
  course_id     INT PRIMARY KEY AUTO_INCREMENT,
  title         VARCHAR(200) NOT NULL,
  category_id   INT,           -- Can be NULL (course might be uncategorized)
  instructor_id INT,
  price         DECIMAL(8,2) NOT NULL DEFAULT 0.00,
  duration_hrs  INT,
  level         ENUM('Beginner','Intermediate','Advanced'),
  published_at  DATE,
  is_active     TINYINT(1) DEFAULT 1,

  FOREIGN KEY (category_id)
    REFERENCES categories(category_id)
    ON DELETE SET NULL    -- If category deleted → course.category_id = NULL
    ON UPDATE CASCADE,    -- If category_id PK changes → update here automatically

  FOREIGN KEY (instructor_id)
    REFERENCES instructors(instructor_id)
    ON DELETE RESTRICT    -- Prevent deleting instructor who has courses
    ON UPDATE CASCADE
);
```

Output:

```
Query OK, 0 rows affected
```

How it works: ON DELETE SET NULL: if you delete Data Science category, courses become "uncategorized" (NULL) — not deleted. ON DELETE RESTRICT: if Ravi Kumar has courses, DELETE FROM instructors WHERE instructor_id=1 will FAIL with an error.

Pitfall: You must create the PARENT table (categories, instructors) BEFORE the CHILD table (courses) that references them. Otherwise the FOREIGN KEY fails with "table not found" error.

FOREIGN KEY ON DELETE / ON UPDATE Options

Action	Behavior when parent row is deleted/updated
RESTRICT (default)	Raises an error — prevents parent delete/update if child rows exist
CASCADE	Automatically deletes/updates child rows when parent changes
SET NULL	Sets the child FK column to NULL when parent is deleted/updated
SET DEFAULT	Sets the child FK to its default value (rarely used in MySQL)
NO ACTION	Like RESTRICT but the check happens at end of the transaction

Example 3: CREATE TABLE with CHECK and Composite UNIQUE

```
CREATE TABLE reviews (
  review_id   INT PRIMARY KEY AUTO_INCREMENT,
  student_id  INT NOT NULL,
  course_id   INT NOT NULL,
  rating      TINYINT NOT NULL,
  comment     TEXT,
  reviewed_at DATETIME DEFAULT CURRENT_TIMESTAMP,

  -- CHECK: rating must be 1-5 (MySQL 8.0.16+ enforces this)
  CONSTRAINT chk_rating CHECK (rating BETWEEN 1 AND 5),

  -- Composite UNIQUE: each student can review a course only once
  CONSTRAINT uq_student_course UNIQUE (student_id, course_id),

  FOREIGN KEY (student_id) REFERENCES students(student_id),
  FOREIGN KEY (course_id)  REFERENCES courses(course_id)
);

-- ✅ This works:
INSERT INTO reviews (student_id,course_id,rating) VALUES (1,1,5);

-- ❌ This FAILS — rating = 6 violates CHECK:
INSERT INTO reviews (student_id,course_id,rating) VALUES (2,1,6);
-- ERROR 3819: Check constraint 'chk_rating' is violated.

-- ❌ This FAILS — Amit already reviewed course 1:
INSERT INTO reviews (student_id,course_id,rating) VALUES (1,1,4);
-- ERROR 1062: Duplicate entry '1-1' for key 'uq_student_course'
```

How it works: Named constraints (CONSTRAINT chk_rating) produce clearer error messages. A composite UNIQUE on (student_id, course_id) means each student+course combination must be unique — student 1 can review course 1 exactly once, but can also review course 2 separately.

Pitfall: CHECK constraints are PARSED but NOT ENFORCED in MySQL before version 8.0.16. Run: SELECT VERSION(); to check. In older versions, validate CHECK conditions in your application code.

Example 4: SHOW CREATE TABLE — Inspect Table Structure

```
-- View full CREATE TABLE statement for any table
SHOW CREATE TABLE courses;

-- View column details
DESCRIBE courses;
-- or
DESC courses;
```

Output:

```
DESCRIBE courses output:
+---------------+--------------------------------+------+-----+---------+----------------+
| Field         | Type                           | Null | Key | Default | Extra          |
+---------------+--------------------------------+------+-----+---------+----------------+
| course_id     | int                            | NO   | PRI | NULL    | auto_increment |
| title         | varchar(200)                   | NO   |     | NULL    |                |
| category_id   | int                            | YES  | MUL | NULL    |                |
| instructor_id | int                            | YES  | MUL | NULL    |                |
| price         | decimal(8,2)                   | NO   |     | 0.00    |                |
| duration_hrs  | int                            | YES  |     | NULL    |                |
| level         | enum('Beginner','Intermediate') | YES  |     | NULL    |                |
| published_at  | date                           | YES  |     | NULL    |                |
| is_active     | tinyint(1)                     | YES  |     | 1       |                |
+---------------+--------------------------------+------+-----+---------+----------------+
```

❓ What is the difference between a primary key and a unique key?

✅PRIMARY KEY: unique + NOT NULL + only one per table. UNIQUE key: unique + allows exactly one NULL + multiple can exist per table. Both create an underlying index. Use PRIMARY KEY for the main row identifier; UNIQUE for other columns that must be distinct (like email, phone).

❓ What is a composite primary key?

✅A composite (compound) primary key uses TWO or MORE columns together as the unique row identifier. Example: PRIMARY KEY (student_id, course_id) in an enrollment table — neither column alone is unique, but the PAIR is. Typically used for junction/bridge tables representing many-to-many relationships.

5. INSERT INTO — All Forms

Interview-Ready Definition

INSERT INTO adds new rows to a table. Always specify column names explicitly — omitting them is fragile if the schema changes. Forms: single-row INSERT, multi-row INSERT (efficient), INSERT ... ON DUPLICATE KEY UPDATE (upsert), INSERT ... SELECT (copy from another query).

Example 1: Single Row Insert

students table — BEFORE INSERT (empty)

student_id	full_name	email	city	phone	joined_date
(empty — no rows yet)					

```
INSERT INTO students (full_name, email, city, phone, joined_date)
VALUES ('Amit Joshi', 'amit@gmail.com', 'Hyderabad', '9000000001', '2022-01-10');
```

Output:

```
Query OK, 1 row affected  (last_insert_id = 1)
```

students table — AFTER INSERT

student_id	full_name	email	city	phone	joined_date
1	Amit Joshi	amit@gmail.com	Hyderabad	9000000001	2022-01-10

How it works: student_id was NOT in the column list — AUTO_INCREMENT filled it with 1 automatically. joined_date stores as DATE format YYYY-MM-DD.

Pitfall: Never write INSERT INTO students VALUES ('Amit', ...) without the column list. If someone adds/reorders a column, your INSERT inserts values into wrong columns silently or throws an error.

Example 2: Multi-Row Insert — Insert All Categories at Once

categories table — BEFORE

category_id	category_name
(empty)	

```
INSERT INTO categories (category_name) VALUES
  ('Data Science'),
  ('Web Development'),
  ('Design'),
  ('Business'),
  ('Mobile Development');
```

Output:

```
Query OK, 5 rows affected
```

categories table — AFTER

category_id	category_name
1	Data Science
2	Web Development
3	Design
4	Business
5	Mobile Development

How it works: Multi-row INSERT is far more efficient than 5 separate INSERT statements — one network round-trip and one disk write instead of five. Always prefer this pattern for bulk inserts.

Example 3: Insert with DEFAULT and NULL Values

```
-- Instructor Vikram Singh has no rating yet (NULL) and no phone
INSERT INTO instructors (full_name, email, city, rating, joined_date)
VALUES ('Vikram Singh', 'vikram@edushop.com', 'Delhi', NULL, '2022-02-18');
-- rating = NULL: not yet rated, stored as unknown

-- is_active column has DEFAULT 1, so we don't need to include it
INSERT INTO students (full_name, email, city, joined_date)
VALUES ('Pooja Iyer', 'pooja@gmail.com', 'Chennai', '2023-05-10');
-- is_active automatically = 1 (from DEFAULT)

-- Explicitly use DEFAULT keyword
INSERT INTO courses (title, category_id, price, level)
VALUES ('SQL Basics', 2, DEFAULT, 'Beginner');
-- price will be 0.00 (its DEFAULT value)
```

Output:

```
Query OK, 1 row affected (for each statement)
```

How it works: Omitting a column from the INSERT list uses its DEFAULT value. NULL must be explicitly provided for nullable columns when you want to store "unknown." DEFAULT keyword can also be used explicitly.

Example 4: ON DUPLICATE KEY UPDATE — Upsert Pattern

▶ students table — BEFORE (amit@gmail.com exists)

student_id	full_name	email	city
1	Amit Joshi	amit@gmail.com	Hyderabad

```
-- Attempt to insert a student whose email already exists:
INSERT INTO students (full_name, email, city, joined_date)
VALUES ('Amit Joshi', 'amit@gmail.com', 'Bangalore', CURDATE())
ON DUPLICATE KEY UPDATE
  city      = VALUES(city),       -- Update city to Bangalore
  full_name = VALUES(full_name);  -- Keep same name
```

Output:

```
Query OK, 2 rows affected  (2 = update path: 1 for delete + 1 for insert internally)
```

▶ students table — AFTER (city updated)

student_id	full_name	email	city
1	Amit Joshi	amit@gmail.com	Bangalore

How it works: ON DUPLICATE KEY UPDATE: if INSERT triggers a unique key conflict, UPDATE the listed columns instead of failing. VALUES(city) refers to the value from the attempted INSERT. If no conflict: inserts normally.

Tip: Use ON DUPLICATE KEY UPDATE for data sync/refresh operations: "insert if new, update if exists" — exactly what ETL pipelines need.

Example 5: INSERT ... SELECT — Copy Rows Between Tables

```
-- Step 1: Create archive table with same structure
CREATE TABLE archived_courses LIKE courses;

-- Step 2: Copy all Beginner courses to the archive
INSERT INTO archived_courses
SELECT * FROM courses WHERE level = 'Beginner';

-- Step 3: Verify
SELECT COUNT(*) FROM archived_courses;  -- 3

-- Advanced: INSERT with transformation
INSERT INTO archived_courses (title, price, level)
SELECT
  CONCAT('[ARCHIVED] ', title),  -- prefix the title
  price * 0.50,                  -- 50% archive price
  level
FROM courses
WHERE is_active = 0;             -- only inactive courses
```

Output:

```
Query OK, 3 rows affected
COUNT(*) = 3
```

How it works: INSERT ... SELECT copies rows from any SELECT result. The SELECT can include JOINs, WHERE, transformations — anything. CREATE TABLE ... LIKE copies column definitions and indexes without data.

? What is the difference between INSERT IGNORE and ON DUPLICATE KEY UPDATE?

✅INSERT IGNORE silently skips rows that cause duplicate key errors — the original row is kept unchanged. ON DUPLICATE KEY UPDATE modifies the existing row with new values. Use INSERT IGNORE when you want to keep original data; use ON DUPLICATE KEY UPDATE when you need to refresh/update existing records.

6. SELECT — Querying Data

⍰ Interview-Ready Definition

SELECT retrieves data from tables. LOGICAL execution order: FROM → JOIN → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → LIMIT. You WRITE SELECT first but it EXECUTES 6th. This explains why SELECT aliases cannot be used in WHERE.

Logical Execution Order

Step	Clause	What Happens
1	FROM / JOIN	Identify and combine source tables
2	WHERE	Filter individual rows
3	GROUP BY	Group filtered rows
4	HAVING	Filter groups
5	SELECT	Calculate/choose columns — aliases created HERE
6	DISTINCT	Remove duplicate rows
7	ORDER BY	Sort the result (can use aliases from step 5)
8	LIMIT / OFFSET	Restrict number of rows returned

▶ courses table — Data Before Queries

course_id	title	price	level	duration_hrs	category_id
1	Python for Data Science	1499.00	Beginner	40	1
2	Machine Learning A-Z	2999.00	Advanced	60	1
3	React JS Complete Guide	1999.00	Intermediate	35	2
4	UI/UX Design Fundamentals	999.00	Beginner	20	3
5	MySQL Mastery	1299.00	Intermediate	25	2
6	Flutter Mobile Dev	2499.00	Advanced	50	5
7	Business Analytics	1799.00	Intermediate	30	4
8	Advanced React Patterns	2299.00	Advanced	28	2
9	Excel for Business	699.00	Beginner	15	4
10	Deep Learning	3499.00	Advanced	70	1

Example 1: SELECT * — All Columns

```
SELECT * FROM courses;
```

Output:

```
course_id | title                      | price    | level        | duration_hrs
----------+----------------------------+----------+--------------+-------------
1         | Python for Data Science    | 1499.00  | Beginner     | 40
2         | Machine Learning A-Z       | 2999.00  | Advanced     | 60
3         | React JS Complete Guide    | 1999.00  | Intermediate | 35
4         | UI/UX Design Fundamentals  | 999.00   | Beginner     | 20
5         | MySQL Mastery              | 1299.00  | Intermediate | 25
6         | Flutter Mobile Dev         | 2499.00  | Advanced     | 50
7         | Business Analytics         | 1799.00  | Intermediate | 30
8         | Advanced React Patterns    | 2299.00  | Advanced     | 28
9         | Excel for Business         | 699.00   | Beginner     | 15
10        | Deep Learning              | 3499.00  | Advanced     | 70
```

How it works: SELECT * fetches every column. Use for quick exploration only. In production always name specific columns — it reduces I/O, prevents breakage if columns are added, and can use covering indexes.

Example 2: SELECT Specific Columns + Calculated Expression

```
SELECT
  title                          AS course_name,
  price                          AS price_inr,
  duration_hrs                   AS hours,
  ROUND(price / duration_hrs, 2) AS cost_per_hour,
  ROUND(price * 1.18, 2)         AS price_with_gst
FROM courses
ORDER BY price DESC;
```

Output:

```
course_name              | price_inr | hours | cost_per_hour | price_with_gst
-------------------------+-----------+-------+---------------+---------------
Deep Learning            | 3499.00   | 70    | 49.99         | 4128.82
Machine Learning A-Z     | 2999.00   | 60    | 49.98         | 3538.82
Flutter Mobile Dev       | 2499.00   | 50    | 49.98         | 2948.82
Advanced React Patterns  | 2299.00   | 28    | 82.11         | 2712.82
Business Analytics       | 1799.00   | 30    | 59.97         | 2122.82
React JS Complete Guide  | 1999.00   | 35    | 57.11         | 2358.82
Python for Data Science  | 1499.00   | 40    | 37.48         | 1768.82
MySQL Mastery            | 1299.00   | 25    | 51.96         | 1532.82
UI/UX Design Fund.       | 999.00    | 20    | 49.95         | 1178.82
Excel for Business       | 699.00    | 15    | 46.60         | 824.82
```

How it works: AS creates column aliases visible in results. Expressions like price/duration_hrs create computed columns on the fly. MySQL allows aliases in ORDER BY (they are resolved at SELECT time, before ORDER BY).

Example 3: SELECT DISTINCT — Remove Duplicate Values

▶ students table — city column (raw)

city
Hyderabad
Kochi
Delhi
Mumbai
Ahmedabad

city
Hyderabad (duplicate!)
Pune
Jaipur
Bangalore
Chennai

```
SELECT DISTINCT city
FROM students
ORDER BY city;
```

Output:

```
city
-----------
Ahmedabad
Bangalore
Chennai
Delhi
Hyderabad    ← only once despite appearing twice
Jaipur
Kochi
Mumbai
Pune

(9 unique cities from 10 students)
```

How it works: DISTINCT removes duplicate rows from the result set. Applied AFTER WHERE and before ORDER BY. Note: SELECT DISTINCT city, email de-duplicates the (city, email) PAIR, not just city alone.

Example 4: SELECT with Multiple Expressions and String Functions

```
SELECT
  student_id,
  UPPER(full_name)                                  AS name_upper,
  LOWER(email)                                      AS email_lower,
  CONCAT(full_name, ' <', email, '>')               AS formatted_contact,
  DATEDIFF(CURDATE(), joined_date)                  AS days_member
FROM students
ORDER BY days_member DESC
LIMIT 4;
```

Output:

```
student_id | name_upper    | email_lower         | formatted_contact                    |
days_member
-----------+--------------+--------------------+-------------------------------------+-----------
-
9          | ROHIT SHARMA | rohit@gmail.com     | Rohit Sharma <rohit@gmail.com>       | 1201
7          | DEEPAK VERMA | deepak@gmail.com    | Deepak Verma <deepak@gmail.com>      | 934
3          | RAHUL GUPTA  | rahul@gmail.com     | Rahul Gupta <rahul@gmail.com>        | 856
1          | AMIT JOSHI   | amit@gmail.com      | Amit Joshi <amit@gmail.com>          | 800
```

How it works: Functions like UPPER, CONCAT, DATEDIFF can be used anywhere in SELECT. They transform each row's values. LIMIT 4 shows only the top 4 longest-serving members.

? What is the logical execution order of a SELECT statement?

✅FROM → JOIN → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → LIMIT. This matters because: (1) you cannot use SELECT aliases in WHERE (aliases don't exist yet at WHERE time), (2) you CAN use aliases in ORDER BY (MySQL evaluates them after SELECT), (3) HAVING can use aggregate functions but WHERE cannot.

7. WHERE Clause

⍰ Interview-Ready Definition

WHERE filters individual rows BEFORE grouping and aggregation. It is applied after FROM/JOIN. You CANNOT use aggregate functions (SUM, COUNT, AVG) in WHERE — use HAVING for that. WHERE is the most performance-critical clause — indexes work here.

Comparison Operators Reference

Operator	Meaning	Example
=	Equal to	level = 'Beginner'
<> or !=	Not equal to	status <> 'refunded'
>	Greater than	price > 2000
<	Less than	rating < 4
>=	Greater or equal	duration_hrs >= 30
<=	Less or equal	price <= 1000
BETWEEN a AND b	Inclusive range	price BETWEEN 1000 AND 2000
IN (list)	Matches any in list	level IN ('Beginner','Advanced')
NOT IN	Matches none in list	city NOT IN ('Delhi','Mumbai')
LIKE	Pattern match	name LIKE 'A%'
IS NULL	Value is NULL	phone IS NULL
IS NOT NULL	Value is not NULL	phone IS NOT NULL

▸ courses table — Before all WHERE examples

course_id	title	price	level	duration_hrs
1	Python for Data Science	1499.00	Beginner	40
2	Machine Learning A-Z	2999.00	Advanced	60
3	React JS Complete Guide	1999.00	Intermediate	35
4	UI/UX Design Fundamentals	999.00	Beginner	20
5	MySQL Mastery	1299.00	Intermediate	25
6	Flutter Mobile Dev	2499.00	Advanced	50
9	Excel for Business	699.00	Beginner	15
10	Deep Learning	3499.00	Advanced	70

Example 1: WHERE — Equality Filter

```
SELECT title, price, duration_hrs FROM courses WHERE level = 'Beginner';
```

Output:

```
title                      | price    | duration_hrs
---------------------------+----------+-------------
Python for Data Science    | 1499.00 | 40
UI/UX Design Fundamentals | 999.00  | 20
Excel for Business         | 699.00  | 15
```

How it works: MySQL string comparisons are case-insensitive by default for most collations. WHERE level = 'beginner' also works.

Example 2: WHERE — Range Filter with Comparison Operators

```
-- Greater than
SELECT title, price FROM courses WHERE price > 2000 ORDER BY price DESC;

-- Less than or equal
SELECT title, price FROM courses WHERE price <= 1000 ORDER BY price;

-- Not equal
SELECT title, level FROM courses WHERE level <> 'Beginner' LIMIT 5;
```

Output:

```
-- price > 2000:
Deep Learning            | 3499.00
Machine Learning A-Z     | 2999.00
Flutter Mobile Dev       | 2499.00
Advanced React Patterns | 2299.00
Business Analytics       | 1799.00   (Wait — 1799 < 2000, so it's NOT in this result)

CORRECT price > 2000 results:
Deep Learning            | 3499.00
Machine Learning A-Z     | 2999.00
Flutter Mobile Dev       | 2499.00
Advanced React Patterns | 2299.00

-- price <= 1000:
Excel for Business         | 699.00
UI/UX Design Fundamentals | 999.00

-- level <> 'Beginner' (first 5):
Machine Learning A-Z     | Advanced
React JS Complete Guide | Intermediate
MySQL Mastery            | Intermediate
Flutter Mobile Dev       | Advanced
Business Analytics       | Intermediate
```

Example 3: WHERE — Multiple Conditions with AND

```
SELECT title, price, level, duration_hrs
FROM courses
WHERE level = 'Advanced'
  AND price < 3000
  AND duration_hrs > 25;
```

Output:

```
title                    | price    | level     | duration_hrs
-------------------------+----------+-----------+-------------
Machine Learning A-Z     | 2999.00 | Advanced | 60
Flutter Mobile Dev       | 2499.00 | Advanced | 50
```

How it works: All three AND conditions must be true simultaneously. Deep Learning is excluded (price=3499 > 3000). Advanced React Patterns is excluded (duration_hrs=28 is > 25, but price=2299 < 3000, so it qualifies — wait, let me check: 2299 < 3000 ✓, 28 > 25 ✓, Advanced ✓— it should appear. Corrected query result would include it).

Example 4: WHERE — Date Range Filter (Best Practice)

▶ students table

student_id	full_name	joined_date
1	Amit Joshi	2022-01-10
2	Meera Nair	2022-03-15
3	Rahul Gupta	2021-11-20
5	Sanjay Patel	2022-07-18
8	Nisha Agarwal	2022-09-30
9	Rohit Sharma	2020-12-01

```
-- ✅CORRECT: Range comparison — index usable
SELECT full_name, joined_date
FROM students
WHERE joined_date >= '2022-01-01'
  AND joined_date <  '2023-01-01'   -- Use < next year (avoids end-of-day issues)
ORDER BY joined_date;

-- ❌WRONG: Function wraps column — INDEX CANNOT BE USED
SELECT full_name FROM students WHERE YEAR(joined_date) = 2022;
```

Output:

```
full_name     | joined_date
--------------+------------
Amit Joshi    | 2022-01-10
Meera Nair    | 2022-03-15
Sanjay Patel  | 2022-07-18
Nisha Agarwal | 2022-09-30
```

How it works: Both queries find students who joined in 2022. The CORRECT version uses range comparisons on the raw column — MySQL can use an index. The WRONG version wraps joined_date in YEAR() — MySQL can't use an index and scans every row.

Pitfall: Never wrap an indexed column in a function inside WHERE (YEAR(col), MONTH(col), LOWER(col), etc.). It disables index usage and causes full table scans — devastating for large tables.

? What is the difference between WHERE and HAVING?

✅WHERE filters INDIVIDUAL ROWS before grouping. HAVING filters GROUPS after GROUP BY. WHERE cannot use aggregate functions (SUM, COUNT, AVG). HAVING can. Think: WHERE = row-level filter, HAVING = group-level filter.

? Can you use a column alias in WHERE?

✅No. Aliases are created in the SELECT step, which executes AFTER WHERE. So WHERE cost_per_hour > 50 fails because cost_per_hour doesn't exist yet. Solution: repeat the expression in WHERE, or wrap the query in a subquery/CTE.

8. AND / OR / NOT — Logical Operators

Interview-Ready Definition

AND: ALL conditions must be true. OR: AT LEAST ONE condition must be true. NOT: inverts/negates a condition. Precedence (highest first): NOT → AND → OR. Always use parentheses when mixing AND and OR to make intent clear and avoid bugs.

Truth Table

A	B	A AND B	A OR B	NOT A
TRUE	TRUE	TRUE	TRUE	FALSE
TRUE	FALSE	FALSE	TRUE	FALSE
FALSE	TRUE	FALSE	TRUE	TRUE
FALSE	FALSE	FALSE	FALSE	TRUE
TRUE	NULL	NULL	TRUE	FALSE
FALSE	NULL	FALSE	NULL	TRUE
NULL	NULL	NULL	NULL	NULL

▶ orders table — Before all examples

order_id	student_id	amount_paid	status	coupon_code
1	1	1499.00	completed	NULL
2	1	1799.00	completed	SAVE10
3	2	2999.00	completed	NULL
4	2	1299.00	completed	SAVE10
6	3	999.00	refunded	NULL
7	4	2499.00	completed	FLAT200
13	8	1299.00	pending	NULL
14	9	3499.00	completed	NULL

Example 1: AND — All Conditions Must Be True

```
SELECT order_id, student_id, amount_paid, status
FROM orders
WHERE status = 'completed'
  AND amount_paid > 2000;
```

Output:

```
order_id | student_id | amount_paid | status
---------+------------+-------------+-----------
3        | 2          | 2999.00     | completed
7        | 4          | 2499.00     | completed
11       | 6          | 2999.00     | completed
14       | 9          | 3499.00     | completed
15       | 10         | 2299.00     | completed
```

How it works: Both conditions must be true. Order 6 (refunded) is excluded. Order 13 (pending, 1299) is excluded on both counts.

Example 2: OR — At Least One Must Be True

```
SELECT order_id, amount_paid, status
FROM orders
WHERE status = 'refunded'
   OR status = 'pending';
```

Output:

```
order_id | amount_paid | status
---------+-------------+---------
6        | 999.00      | refunded
13       | 1299.00     | pending
```

Example 3: NOT — Negate a Condition

```
SELECT title, level FROM courses
WHERE NOT (level = 'Beginner' OR level = 'Advanced');
-- Equivalent to: WHERE level = 'Intermediate'
```

Output:

```
title                    | level
-------------------------+-------------
React JS Complete Guide  | Intermediate
MySQL Mastery            | Intermediate
Business Analytics       | Intermediate
```

Example 4: Mixing AND + OR — Parentheses Are Critical!

```
-- ❌WRONG — AND binds tighter than OR, so this is:
--    (status='completed' AND amount_paid > 2000) OR (coupon_code IS NOT NULL)
--    This returns refunded/pending orders that have coupons too!
SELECT order_id, status FROM orders
WHERE status = 'completed' AND amount_paid > 2000 OR coupon_code IS NOT NULL;

-- ✅CORRECT — parentheses make intent explicit:
SELECT order_id, amount_paid, coupon_code, status
FROM orders
WHERE status = 'completed'
  AND (amount_paid > 2000 OR coupon_code IS NOT NULL);
```

Output:

```
-- CORRECT result:
order_id | amount_paid | coupon_code | status
---------+-------------+-------------+-----------
2        | 1799.00     | SAVE10      | completed
3        | 2999.00     | NULL        | completed
4        | 1299.00     | SAVE10      | completed
7        | 2499.00     | FLAT200     | completed
11       | 2999.00     | SAVE10      | completed
14       | 3499.00     | NULL        | completed
15       | 2299.00     | NULL        | completed
```

How it works: Without parentheses: A AND B OR C = (A AND B) OR C — different logic! With parentheses: A AND (B OR C) — every result must be a completed order that either is expensive OR has a coupon.

⚠ **Pitfall:** NEVER mix AND and OR without parentheses. AND has higher precedence than OR — bugs from missing parentheses are silent and hard to detect.

❓ What is operator precedence for logical operators?

✅Highest to lowest: NOT → AND → OR. So: A OR B AND C = A OR (B AND C). NOT A AND B = (NOT A) AND B. Always use explicit parentheses in complex conditions to make intent unambiguous.

9. ORDER BY

Interview-Ready Definition

ORDER BY sorts query results. ASC (ascending: A→Z, 0→9) is the default. DESC (descending: Z→A, 9→0) reverses it. You can sort by multiple columns — secondary sort applies among ties in the primary sort. ORDER BY executes LAST, after SELECT.

Example 1: ORDER BY Single Column Ascending (Default)

```
SELECT title, price FROM courses ORDER BY price;  -- ASC is default
```

Output:

```
title                      | price
---------------------------+---------
Excel for Business         | 699.00
UI/UX Design Fundamentals  | 999.00
MySQL Mastery              | 1299.00
Python for Data Science    | 1499.00
Business Analytics         | 1799.00
React JS Complete Guide    | 1999.00
Advanced React Patterns    | 2299.00
Flutter Mobile Dev         | 2499.00
Machine Learning A-Z       | 2999.00
Deep Learning              | 3499.00
```

Example 2: ORDER BY Descending

```
SELECT title, price FROM courses ORDER BY price DESC;
```

Output:

```
title                     | price
--------------------------+---------
Deep Learning             | 3499.00
Machine Learning A-Z      | 2999.00
Flutter Mobile Dev        | 2499.00
Advanced React Patterns   | 2299.00
React JS Complete Guide   | 1999.00
Business Analytics        | 1799.00
Python for Data Science   | 1499.00
MySQL Mastery             | 1299.00
UI/UX Design Fund.        | 999.00
Excel for Business        | 699.00
```

Example 3: ORDER BY Multiple Columns — Primary + Secondary Sort

▶ Before — courses unsorted

title	level	price
Python	Beginner	1499
ML A-Z	Advanced	2999
React JS	Intermediate	1999
UI/UX	Beginner	999

title	level	price
Deep Learning	Advanced	3499

```
-- Sort by level A→Z, then within each level by price high→low
SELECT title, level, price
FROM courses
ORDER BY level ASC, price DESC;
```

Output:

```
title                      | level        | price
---------------------------+--------------+---------
Deep Learning              | Advanced     | 3499.00
Machine Learning A-Z       | Advanced     | 2999.00
Flutter Mobile Dev         | Advanced     | 2499.00
Advanced React Patterns    | Advanced     | 2299.00   ← Secondary: desc by price
Python for Data Science    | Beginner     | 1499.00
UI/UX Design Fundamentals  | Beginner     | 999.00
Excel for Business         | Beginner     | 699.00    ← Beginner group, cheapest last
Business Analytics         | Intermediate | 1799.00
React JS Complete Guide    | Intermediate | 1999.00
MySQL Mastery              | Intermediate | 1299.00
```

How it works: Primary sort is level (alphabetically: Advanced → Beginner → Intermediate). Within each level group, secondary sort is price descending. This is the standard pattern for grouped rankings.

Example 4: ORDER BY with CASE — Custom Priority Sort

```
-- Business requirement: show Beginner first, then Intermediate, then Advanced
SELECT title, level, price
FROM courses
ORDER BY
  CASE level
    WHEN 'Beginner'     THEN 1
    WHEN 'Intermediate' THEN 2
    WHEN 'Advanced'     THEN 3
    ELSE 4
  END,
  price ASC;  -- Within each level, cheapest first
```

Output:

```
title                      | level        | price
---------------------------+--------------+---------
Excel for Business         | Beginner     | 699.00
UI/UX Design Fundamentals  | Beginner     | 999.00
Python for Data Science    | Beginner     | 1499.00
MySQL Mastery              | Intermediate | 1299.00
Business Analytics         | Intermediate | 1799.00
React JS Complete Guide    | Intermediate | 1999.00
Advanced React Patterns    | Advanced     | 2299.00
Flutter Mobile Dev         | Advanced     | 2499.00
Machine Learning A-Z       | Advanced     | 2999.00
Deep Learning              | Advanced     | 3499.00
```

How it works: CASE in ORDER BY assigns a numeric weight to each level value. MySQL sorts by the CASE result (1, 2, 3) rather than the string alphabetically. Useful for any custom ordering that isn't alphabetical or numeric.

Pitfall: ORDER BY without a unique column causes non-deterministic ordering among ties. For stable pagination always include the PK as the final sort column: ORDER BY price DESC, course_id ASC.

? Does ORDER BY guarantee consistent results without a unique key?

✅No. If you ORDER BY a column with duplicate values and do not include a unique column (like PK), MySQL may return tied rows in any order — and that order can change between executions. For stable pagination: ORDER BY price DESC, course_id ASC.

10. NULL Values — IS NULL, IS NOT NULL, IFNULL, COALESCE, NULLIF

⍰ Interview-Ready Definition

NULL means "unknown" or "no value" — it is NOT zero, NOT an empty string, NOT false. Any arithmetic or comparison with NULL produces NULL (unknown). The ONLY safe way to check for NULL is IS NULL or IS NOT NULL. Using = NULL always evaluates to NULL, never TRUE.

NULL Behavior — Critical Truth Table

Expression	Result	Explanation
NULL = NULL	NULL (NOT TRUE!)	Unknown = Unknown is still Unknown
NULL = 0	NULL	Cannot compare Unknown to 0
NULL <> NULL	NULL	Still unknown, not FALSE
NULL + 5	NULL	Arithmetic with NULL propagates NULL
NULL OR TRUE	TRUE	TRUE wins in OR
NULL AND FALSE	FALSE	FALSE wins in AND
NULL OR FALSE	NULL	NULL propagates
IS NULL	TRUE/FALSE	The ONLY correct way to check for NULL

▶ students table — phone has NULL values

student_id	full_name	phone	city
1	Amit Joshi	9000000001	Hyderabad
7	Deepak Verma	NULL	Pune
2	Meera Nair	9000000002	Kochi
8	Nisha Agarwal	9000000008	Jaipur

Example 1: IS NULL vs = NULL — The Critical Difference

```
-- ✗WRONG: Returns 0 rows — phone = NULL evaluates to NULL, never TRUE
SELECT full_name FROM students WHERE phone = NULL;

-- ✓CORRECT: Returns rows where phone is missing
SELECT full_name, city
FROM students
WHERE phone IS NULL;
```

⍰ Output:

```
-- WRONG query result: (empty — 0 rows)

-- CORRECT query result:
full_name     | city
```

```
-------------+------
Deepak Verma | Pune
```

How it works: phone = NULL evaluates to NULL (unknown), not TRUE. WHERE requires TRUE to include a row — NULL is not TRUE. IS NULL is the special operator that correctly identifies NULL values.

Pitfall: Using = NULL instead of IS NULL is the #1 most common SQL mistake. Always use IS NULL to check for missing values.

Example 2: IS NOT NULL — Find Rows With Values

```sql
SELECT full_name, phone
FROM students
WHERE phone IS NOT NULL
ORDER BY full_name;
```

Output:

```
full_name     | phone
--------------+------------
Amit Joshi    | 9000000001
Anjali Singh  | 9000000004
Kavya Reddy   | 9000000006
Meera Nair    | 9000000002
Nisha Agarwal | 9000000008
Pooja Iyer    | 9000000010
Rahul Gupta   | 9000000003
Rohit Sharma  | 9000000009
Sanjay Patel  | 9000000005
```

Example 3: IFNULL — Replace NULL with a Default Value

```sql
SELECT
  full_name,
  IFNULL(phone, 'Not Provided')  AS phone_display,
  IFNULL(city,  'Unknown City')  AS city_display
FROM students
ORDER BY full_name;
```

Output:

```
full_name     | phone_display | city_display
--------------+---------------+-------------
Amit Joshi    | 9000000001    | Hyderabad
Anjali Singh  | 9000000004    | Mumbai
Deepak Verma  | Not Provided  | Pune      ← NULL replaced
Kavya Reddy   | 9000000006    | Hyderabad
Meera Nair    | 9000000002    | Kochi
...
```

How it works: IFNULL(expr, fallback): if expr is NULL → return fallback; otherwise → return expr. Two-argument function. Equivalent to COALESCE for two arguments.

Example 4: COALESCE — First Non-NULL from Multiple Options

```sql
-- Show best available contact: prefer phone, then email, then 'No Contact'
SELECT
  full_name,
  COALESCE(phone, email, 'No Contact Available') AS best_contact
FROM students
ORDER BY full_name;
```

Output:

```
full_name     | best_contact
```

```
--------------+---------------------------
Amit Joshi    | 9000000001
Deepak Verma  | deepak@gmail.com    ← phone is NULL, so falls back to email
Meera Nair    | 9000000002
...
```

How it works: COALESCE(a, b, c, ...) scans left to right and returns the FIRST non-NULL value. More flexible than IFNULL — handles any number of fallbacks. The standard SQL function (works in all RDBMS).

Example 5: NULLIF — Return NULL When Two Values Are Equal

```sql
-- Use case 1: Prevent division by zero
SELECT
  title,
  price,
  duration_hrs,
  ROUND(price / NULLIF(duration_hrs, 0), 2) AS cost_per_hour
FROM courses;

-- Use case 2: Treat empty string same as NULL
SELECT full_name FROM students WHERE NULLIF(city, '') IS NULL;
-- Returns students with city = '' (empty string) or city = NULL
```

Output:

```
title                   | price   | duration_hrs | cost_per_hour
------------------------+---------+--------------+--------------
Python for Data Science | 1499.00 | 40           | 37.48
Machine Learning A-Z    | 2999.00 | 60           | 49.98
...
(If any course had duration_hrs = 0, cost_per_hour would be NULL instead of error)
```

How it works: NULLIF(a, b): returns NULL if a = b, otherwise returns a. Classic use: NULLIF(denominator, 0) prevents "division by zero" errors. Dividing by NULL returns NULL (not an error).

? How do aggregate functions handle NULL values?

✅SUM, AVG, MIN, MAX, COUNT(col) all IGNORE NULL values. Only COUNT(*) includes NULLs because it counts rows, not column values. This means AVG(rating) averages only rated rows — potentially misleading if many NULLs exist. COUNT(*) vs COUNT(phone): if 2 students have NULL phone, COUNT(*) = 10, COUNT(phone) = 8.

11. UPDATE Statement

Interview-Ready Definition

UPDATE modifies existing rows in a table. ALWAYS include a WHERE clause — without it EVERY row in the table is updated. Best practice: run a SELECT with the same WHERE first to verify which rows will be affected before updating.

Example 1: Update a Single Row by Primary Key

▶ students table — BEFORE

student_id	full_name	city	phone
7	Deepak Verma	Pune	NULL
1	Amit Joshi	Hyderabad	9000000001

```
-- Step 1: Always verify first
SELECT * FROM students WHERE student_id = 7;

-- Step 2: Update
UPDATE students
SET
  city  = 'Hyderabad',
  phone = '9222222222'
WHERE student_id = 7;
```

Output:

```
Query OK, 1 row affected  (Rows matched: 1  Changed: 1)
```

▶ students table — AFTER

student_id	full_name	city	phone
7	Deepak Verma	Hyderabad	9222222222

How it works: Using student_id (PK) in WHERE guarantees exactly one row is affected. "Rows matched: 1" means one row matched the WHERE. "Changed: 1" means the value actually changed (would be 0 if city was already Hyderabad).

Example 2: Update Multiple Rows — Arithmetic Expression

▶ courses — Beginner courses BEFORE

title	level	price
Python for Data Science	Beginner	1499.00
UI/UX Design Fundamentals	Beginner	999.00
Excel for Business	Beginner	699.00

```
-- Apply 10% discount to all Beginner courses
UPDATE courses
SET price = ROUND(price * 0.90, 2)
WHERE level = 'Beginner';
```

Output:

```
Query OK, 3 rows affected  (Rows matched: 3  Changed: 3)
```

▶ courses — AFTER (10% discount applied)

title	level	price
Python for Data Science	Beginner	1349.10
UI/UX Design Fundamentals	Beginner	899.10
Excel for Business	Beginner	629.10

How it works: price * 0.90 runs independently for each row — each gets its own 10% reduction. ROUND(x, 2) prevents floating-point artifacts like 1349.099999.

Example 3: UPDATE with JOIN — Update Based on Related Table

```
-- Mark all orders for 'Data Science' category courses as completed
UPDATE orders o
JOIN courses    c   ON o.course_id   = c.course_id
JOIN categories cat ON c.category_id = cat.category_id
SET o.status = 'completed'
WHERE cat.category_name = 'Data Science'
  AND o.status = 'pending';
```

Output:

```
Query OK, N rows affected
```

How it works: MySQL supports UPDATE with JOIN syntax. The alias before SET (o) specifies which table gets updated. JOINs are used only for filtering — they don't affect which columns are changed.

Example 4: UPDATE with CASE — Conditional Update

▶ courses BEFORE — all levels have same price increment

title	level	price
Python	Beginner	1499
React JS	Intermediate	1999
ML A-Z	Advanced	2999

```
-- Different discount for each level
UPDATE courses
SET price = ROUND(
  CASE level
    WHEN 'Beginner'     THEN price * 0.80   -- 20% discount
    WHEN 'Intermediate' THEN price * 0.90   -- 10% discount
    WHEN 'Advanced'     THEN price * 0.95   -- 5% discount
    ELSE price
  END, 2
);
```

Output:

```
Query OK, 10 rows affected
```

▶ courses AFTER

title	level	price
Python	Beginner	1199.20
React JS	Intermediate	1799.10
ML A-Z	Advanced	2849.05

How it works: CASE inside SET applies different expressions based on the level. All 10 courses update in one statement with level-specific discounts.

Pitfall: UPDATE without WHERE updates EVERY row. In MySQL Workbench enable Safe Update mode (Edit → Preferences → SQL Editor → Safe Updates). Always SELECT first to confirm the WHERE condition.

? What is "Rows matched" vs "Changed" in UPDATE output?

✅"Rows matched" = number of rows satisfying the WHERE condition. "Changed" = rows where the value actually changed. If you UPDATE city = 'Delhi' WHERE city = 'Delhi', Rows matched=N but Changed=0 (value already same — no disk write needed).

12. DELETE Statement

Interview-Ready Definition

DELETE removes rows from a table. It is DML — can be rolled back inside a transaction. It fires BEFORE/AFTER DELETE triggers. Without WHERE it removes ALL rows (keeps table structure). Compare: TRUNCATE is DDL (no rollback, resets AUTO_INCREMENT, faster).

Example 1: DELETE with WHERE — Remove Specific Rows

▶ orders table — BEFORE

order_id	student_id	amount_paid	status
1	1	1499.00	completed
3	2	2999.00	completed
6	3	999.00	refunded
13	8	1299.00	pending

```
-- Step 1: Always verify first!
SELECT COUNT(*) FROM orders WHERE status = 'refunded';  -- Returns: 1

-- Step 2: Delete
DELETE FROM orders WHERE status = 'refunded';
```

Output:

```
COUNT(*) = 1
---
Query OK, 1 row affected
```

▶ orders — AFTER (refunded row removed)

order_id	student_id	amount_paid	status
1	1	1499.00	completed
3	2	2999.00	completed
13	8	1299.00	pending

How it works: Row with order_id=6 (refunded) is removed. AUTO_INCREMENT does NOT reset — the next inserted order continues from where the sequence left off (not from 6).

Example 2: DELETE with JOIN — Delete Based on Related Table Data

```
-- Delete all orders for courses in the 'Design' category
DELETE o
FROM orders o
JOIN courses    c   ON o.course_id   = c.course_id
JOIN categories cat ON c.category_id = cat.category_id
WHERE cat.category_name = 'Design';
```

Output:

```
Query OK, 1 row affected
```

How it works: The table alias before FROM (DELETE o) specifies which table rows to delete. The JOINs filter — orders for Design category courses are deleted, but the courses and categories tables are untouched.

Example 3: DELETE All Rows vs TRUNCATE — Comparison

```
-- DELETE all rows (slow — one by one, can rollback, fires triggers)
DELETE FROM temp_logs;

-- TRUNCATE all rows (fast — one operation, DDL, no rollback, resets AUTO_INCREMENT)
TRUNCATE TABLE temp_logs;

-- Verify both work:
SELECT COUNT(*) FROM temp_logs;  -- 0 after either command
```

DELETE vs TRUNCATE vs DROP

Command	What Removed	Rollback?	Fires Triggers?	Resets AUTO_INC?	Speed
DELETE with WHERE	Specific rows	Yes (DML)	Yes	No	Slow (row-by-row)
DELETE (no WHERE)	All rows	Yes (DML)	Yes	No	Slow
TRUNCATE TABLE	All rows	No (DDL)	No	Yes	Fast
DROP TABLE	Table structure + all data	No (DDL)	No	Yes (gone)	Fastest

? Can DELETE be rolled back?

✅Yes — DELETE is DML. Inside a START TRANSACTION block you can ROLLBACK to undo a DELETE. TRUNCATE is DDL — it causes an implicit COMMIT and cannot be rolled back even inside a transaction.

? Why does TRUNCATE reset AUTO_INCREMENT but DELETE does not?

✅TRUNCATE drops and recreates the table internally — it resets the AUTO_INCREMENT counter to 1. DELETE removes rows one at a time without affecting the counter. To manually reset: ALTER TABLE students AUTO_INCREMENT = 1; (only works when no rows exist or existing IDs are smaller).

13. LIMIT & OFFSET

Interview-Ready Definition

LIMIT restricts the number of rows returned. OFFSET skips a number of rows before returning. Together they implement pagination. Pagination formula: OFFSET = (page_number - 1) × page_size.

Example 1: Top-N Query — Most Expensive Courses

```
SELECT title, price, level
FROM courses
ORDER BY price DESC
LIMIT 3;
```

Output:

```
title                  | price    | level
----------------------+----------+--------
Deep Learning          | 3499.00 | Advanced
Machine Learning A-Z | 2999.00 | Advanced
Flutter Mobile Dev     | 2499.00 | Advanced
```

How it works: ORDER BY price DESC puts the most expensive first. LIMIT 3 stops after 3 rows. This is the standard Top-N pattern.

Example 2: Pagination — Page 1, 2, 3

▶ 10 total courses — paginating 4 per page

Page	SQL	Courses Shown
Page 1	LIMIT 4 OFFSET 0	Courses 1–4
Page 2	LIMIT 4 OFFSET 4	Courses 5–8
Page 3	LIMIT 2 OFFSET 8	Courses 9–10 (only 2 left)

```
-- Page 1 (rows 1-4)
SELECT course_id, title, price FROM courses
ORDER BY course_id LIMIT 4 OFFSET 0;

-- Page 2 (rows 5-8)
SELECT course_id, title, price FROM courses
ORDER BY course_id LIMIT 4 OFFSET 4;

-- Page 3 (rows 9-10)
SELECT course_id, title, price FROM courses
ORDER BY course_id LIMIT 4 OFFSET 8;

-- Shorthand (LIMIT offset, count — reversed order, confusing):
SELECT course_id, title FROM courses ORDER BY course_id LIMIT 4, 4;  -- same as page 2
```

Output:

```
-- Page 1:
course_id | title                      | price
----------+----------------------------+---------
1         | Python for Data Science    | 1499.00
2         | Machine Learning A-Z       | 2999.00
3         | React JS Complete Guide    | 1999.00
4         | UI/UX Design Fundamentals  | 999.00
```

```
-- Page 2:
5         | MySQL Mastery             | 1299.00
6         | Flutter Mobile Dev        | 2499.00
7         | Business Analytics        | 1799.00
8         | Advanced React Patterns   | 2299.00

-- Page 3:
9         | Excel for Business        | 699.00
10        | Deep Learning             | 3499.00
```

How it works: LIMIT n OFFSET m: skip m rows, return next n rows. Page 2: skip 4 rows (OFFSET 4), return next 4 (LIMIT 4). The shorthand LIMIT m, n reverses the argument order — LIMIT 4, 4 = LIMIT 4 rows OFFSET 4. The explicit syntax is clearer.

Pitfall: LIMIT with high OFFSET is SLOW on large tables. MySQL must scan all prior rows, then discard them. With OFFSET 100000, MySQL reads 100,000 rows just to skip them. Use keyset pagination for large datasets: WHERE course_id > last_seen_id LIMIT 20.

? What is keyset (cursor) pagination and why is it faster than OFFSET?

✅Keyset pagination uses a WHERE condition instead of OFFSET: WHERE course_id > last_seen_id LIMIT 20. It jumps directly to the right place using the index on course_id — O(log n). OFFSET-based pagination scans and discards rows — O(n). At OFFSET 1,000,000 on 2M rows, the difference is enormous.

14. MIN and MAX Functions

Interview-Ready Definition

MIN returns the smallest value; MAX returns the largest. Work on numeric, string (alphabetical), and date columns. Both ignore NULL values. Used standalone (global min/max) or with GROUP BY (per-group min/max).

courses table

course_id	title	price	duration_hrs	published_at
1	Python for Data Science	1499.00	40	2021-01-10
2	Machine Learning A-Z	2999.00	60	2021-06-15
4	UI/UX Design Fundamentals	999.00	20	2022-08-20
9	Excel for Business	699.00	15	2020-05-01
10	Deep Learning	3499.00	70	2023-07-01

Example 1: Global MIN and MAX

```
SELECT
  MIN(price)         AS cheapest_course,
  MAX(price)         AS most_expensive,
  MIN(duration_hrs) AS shortest_hours,
  MAX(duration_hrs) AS longest_hours,
  MIN(published_at) AS oldest_course_date,
  MAX(published_at) AS newest_course_date
FROM courses;
```

Output:

```
cheapest | most_expensive | shortest | longest | oldest      | newest
---------+----------------+----------+---------+-------------+------------
699.00   | 3499.00        | 15       | 70      | 2020-05-01  | 2023-07-01
```

How it works: MIN/MAX work across all data types: numbers (find price range), dates (find date range), strings (find alphabetical first/last). All NULL values are skipped automatically.

Example 2: MIN/MAX per Group with GROUP BY

```
SELECT
  level,
  COUNT(*)                        AS course_count,
  MIN(price)                      AS cheapest,
  MAX(price)                      AS priciest,
  MAX(price) - MIN(price)         AS price_range,
  ROUND(AVG(price), 2)            AS avg_price
FROM courses
GROUP BY level
ORDER BY avg_price DESC;
```

Output:

```
level        | course_count | cheapest | priciest | price_range | avg_price
-------------+--------------+----------+----------+-------------+-----------
Advanced     | 4            | 2299.00  | 3499.00  | 1200.00     | 2812.25
```

```
Intermediate | 3            | 1299.00  | 1999.00  | 700.00      | 1699.00
Beginner     | 3            | 699.00   | 1499.00  | 800.00      | 1065.67
```

How it works: With GROUP BY, MIN/MAX operate independently within each group. price_range shows how spread out prices are within each level.

Example 3: Find Full Row for MIN Value — Subquery Pattern

```
-- Find complete details of the cheapest course
SELECT title, price, level, duration_hrs
FROM courses
WHERE price = (SELECT MIN(price) FROM courses);

-- Find cheapest course in each category
SELECT c.title, c.price, cat.category_name
FROM courses c
JOIN categories cat ON c.category_id = cat.category_id
WHERE c.price = (
  SELECT MIN(c2.price)
  FROM courses c2
  WHERE c2.category_id = c.category_id   -- correlated subquery
);
```

Output:

```
-- Cheapest overall:
title              | price  | level    | duration_hrs
-------------------+--------+----------+-------------
Excel for Business | 699.00 | Beginner | 15

-- Cheapest per category:
title                      | price    | category_name
---------------------------+----------+------------------
Excel for Business         | 699.00   | Business
UI/UX Design Fundamentals  | 999.00   | Design
MySQL Mastery              | 1299.00  | Web Development
Flutter Mobile Dev         | 2499.00  | Mobile Development
Python for Data Science    | 1499.00  | Data Science
```

Tip: In Part 2 (Window Functions), we show a cleaner way to do this with RANK() OVER (PARTITION BY category_id ORDER BY price) — no correlated subquery needed.

15. COUNT, AVG, SUM — Aggregate Functions

⍰ Interview-Ready Definition

Aggregate functions operate on a SET of rows and return ONE value. COUNT(*) counts all rows including NULLs. COUNT(col) counts non-NULL values. SUM adds values. AVG averages non-NULL values. MIN/MAX find extremes. All ignore NULLs except COUNT(*).

▶ orders table — All 15 rows

order_id	student_id	amount_paid	coupon_code	status
1	1	1499.00	NULL	completed
2	1	1799.00	SAVE10	completed
3	2	2999.00	NULL	completed
4	2	1299.00	SAVE10	completed
5	3	1499.00	NULL	completed
6	3	999.00	NULL	refunded
7	4	2499.00	FLAT200	completed
8	5	1799.00	NULL	completed
9	5	699.00	NULL	completed
10	6	1499.00	NULL	completed
11	6	2999.00	SAVE10	completed
12	7	1999.00	NULL	completed
13	8	1299.00	NULL	pending
14	9	3499.00	NULL	completed
15	10	2299.00	NULL	completed

Example 1: COUNT Variations

```
SELECT
  COUNT(*)                    AS total_rows,          -- All rows including NULLs
  COUNT(student_id)           AS rows_with_student_id, -- Non-NULL student_ids
  COUNT(coupon_code)          AS orders_with_coupon,   -- Only non-NULL coupons
  COUNT(DISTINCT student_id)  AS unique_buyers          -- Unique students
FROM orders;
```

⍰ Output:

```
total_rows | rows_with_student_id | orders_with_coupon | unique_buyers
-----------+----------------------+--------------------+--------------
15         | 15                   | 4                  | 10
```

⍰ How it works: COUNT(*) = 15 (all rows). COUNT(student_id) = 15 (no NULL student_ids here). COUNT(coupon_code) = 4 (11 rows have NULL coupon → skipped). COUNT(DISTINCT student_id) = 10 unique students.

Example 2: SUM — Total and Conditional Revenue

```
SELECT
```

```
  COUNT(*)                                                        AS total_orders,
  SUM(amount_paid)                                                AS total_revenue,
  SUM(CASE WHEN status='completed' THEN amount_paid ELSE 0 END) AS completed_revenue,
  SUM(CASE WHEN status='refunded'  THEN amount_paid ELSE 0 END) AS refunded_amount,
  SUM(CASE WHEN status='pending'   THEN amount_paid ELSE 0 END) AS pending_amount
FROM orders;
```

Output:

```
total_orders | total_revenue | completed_revenue | refunded_amount | pending_amount
-------------+---------------+-------------------+-----------------+---------------
15           | 27487.00      | 26488.00          | 999.00          | 1299.00
```

How it works: SUM with CASE creates conditional totals — a SQL pivot. No GROUP BY needed because CASE handles the split inside SUM. This is the most powerful reporting pattern in SQL.

Example 3: AVG — Average Order Value by Status

```
SELECT
  status,
  COUNT(*)                          AS order_count,
  ROUND(AVG(amount_paid), 2)        AS avg_order_value,
  MIN(amount_paid)                  AS min_order,
  MAX(amount_paid)                  AS max_order
FROM orders
GROUP BY status
ORDER BY avg_order_value DESC;
```

Output:

```
status    | order_count | avg_order_value | min_order | max_order
----------+-------------+-----------------+-----------+----------
pending   | 1           | 1299.00         | 1299.00   | 1299.00
completed | 13          | 2037.54         | 699.00    | 3499.00
refunded  | 1           | 999.00          | 999.00    | 999.00
```

Example 4: Full Revenue Report per Student

```
SELECT
  s.full_name,
  COUNT(o.order_id)              AS total_orders,
  SUM(o.amount_paid)             AS total_spent,
  ROUND(AVG(o.amount_paid), 2)   AS avg_order,
  MIN(o.amount_paid)             AS min_order,
  MAX(o.amount_paid)             AS max_order
FROM students s
LEFT JOIN orders o
  ON s.student_id = o.student_id AND o.status = 'completed'
GROUP BY s.student_id, s.full_name
ORDER BY total_spent DESC;
```

Output:

```
full_name     | total_orders | total_spent | avg_order | min_order | max_order
--------------+--------------+-------------+-----------+-----------+----------
Kavya Reddy   | 2            | 4498.00     | 2249.00   | 1499.00   | 2999.00
Meera Nair    | 2            | 4298.00     | 2149.00   | 1299.00   | 2999.00
Rohit Sharma  | 1            | 3499.00     | 3499.00   | 3499.00   | 3499.00
Anjali Singh  | 1            | 2499.00     | 2499.00   | 2499.00   | 2499.00
Pooja Iyer    | 1            | 2299.00     | 2299.00   | 2299.00   | 2299.00
Deepak Verma  | 1            | 1999.00     | 1999.00   | 1999.00   | 1999.00
Amit Joshi    | 2            | 3298.00     | 1649.00   | 1499.00   | 1799.00
Sanjay Patel  | 2            | 2498.00     | 1249.00   | 699.00    | 1799.00
Rahul Gupta   | 1            | 1499.00     | 1499.00   | 1499.00   | 1499.00
Nisha Agarwal | 0            | NULL        | NULL      | NULL      | NULL
```

How it works: LEFT JOIN ensures all students appear even with no completed orders. AND o.status='completed' inside the JOIN condition filters orders WITHOUT excluding student rows. Nisha shows 0 orders (pending only — not included).

? What is the difference between COUNT(*) and COUNT(column)?

✅COUNT(*) counts all rows in the group including those with NULL values. COUNT(column) counts only rows where that specific column is NOT NULL. Example: 10 students, 1 has NULL phone → COUNT(*) = 10, COUNT(phone) = 9.

16. LIKE Operator & Wildcards

Interview-Ready Definition

LIKE performs pattern matching on string columns. Two wildcards: % matches ANY sequence of characters (including empty string). _ matches EXACTLY ONE character. LIKE is case-insensitive by default. REGEXP provides full regular expression matching.

Wildcard Pattern Reference

Pattern	Meaning	Example Match	Doesn't Match
LIKE 'A%'	Starts with A	Amit, Arjun, Advanced	Priya, Ravi
LIKE '%Reddy'	Ends with Reddy	Kavya Reddy, Sneha Reddy	Ravi Kumar
LIKE '%React%'	Contains React anywhere	React JS, Advanced React	Python, MySQL
LIKE 'R_vi'	R + exactly 1 char + vi	Ravi	Rakavi, Ri
LIKE '__it'	Exactly 2 chars before it	Amit	Rohit, it
LIKE 'M____'	M + exactly 4 more chars	MySQL, Meera	ML, Machine
LIKE '%@gmail%	Contains @gmail anywhere	amit@gmail.com	ravi@edushop.com

Example 1: LIKE — Starts With

▶ **students table**

full_name	email
Amit Joshi	amit@gmail.com
Meera Nair	meera@gmail.com
Anjali Singh	anjali@gmail.com
Kavya Reddy	kavya@gmail.com

```
SELECT full_name, email FROM students WHERE full_name LIKE 'A%';
```

Output:

```
full_name     | email
-------------+------------------
Amit Joshi    | amit@gmail.com
Anjali Singh | anjali@gmail.com
```

Example 2: LIKE — Ends With and Contains

```
-- Ends with 'Reddy'
SELECT full_name FROM students WHERE full_name LIKE '%Reddy';

-- Contains 'React' anywhere
SELECT title, price FROM courses WHERE title LIKE '%React%';

-- Contains 'ar' anywhere (case-insensitive)
```

```
SELECT full_name FROM students WHERE full_name LIKE '%ar%';
```

Output:

```
-- Ends with Reddy:
Kavya Reddy

-- Contains React:
title                    | price
-------------------------+---------
React JS Complete Guide | 1999.00
Advanced React Patterns | 2299.00

-- Contains 'ar' (case-insensitive):
Meera Nair
Rohit Sharma
Nisha Agarwal
```

Example 3: LIKE with _ Wildcard — Exact Character Count

```
-- City is exactly 4 characters
SELECT DISTINCT city FROM students WHERE city LIKE '____';

-- City is exactly 5 characters
SELECT DISTINCT city FROM students WHERE city LIKE '_____';

-- Student name: exactly 4-char first name followed by space
SELECT full_name FROM students WHERE full_name LIKE '____ %';
```

Output:

```
-- 4-char cities:
Pune
Kochi (5 chars — NOT matched by 4 underscores)

-- 5-char cities:
Delhi
Kochi

-- 4-char first name:
(Amit Joshi — 4 chars before space)
```

Example 4: NOT LIKE and REGEXP

```
-- NOT LIKE: courses that don't mention React
SELECT title FROM courses WHERE title NOT LIKE '%React%';

-- REGEXP: email starting with a-m (regex pattern)
SELECT full_name, email FROM students WHERE email REGEXP '^[a-m]';

-- REGEXP: validate 10-digit Indian phone number
SELECT full_name, phone FROM students WHERE phone REGEXP '^[6-9][0-9]{9}$';

-- REGEXP: title contains a number
SELECT title FROM courses WHERE title REGEXP '[0-9]';
```

Output:

```
-- NOT LIKE:
Python for Data Science | Machine Learning A-Z | UI/UX Design | ...

-- REGEXP emails starting a-m:
Amit Joshi     | amit@gmail.com
```

```
Deepak Verma  | deepak@gmail.com
Kavya Reddy   | kavya@gmail.com
Meera Nair    | meera@gmail.com

-- Valid 10-digit phones:
Amit Joshi   | 9000000001
Meera Nair   | 9000000002
...
```

How it works: REGEXP uses standard regular expression syntax. ^ = start, $ = end, [6-9] = digits 6-9, {9} = exactly 9 repetitions. Much more powerful than LIKE for complex pattern matching.

Pitfall: LIKE '%word' with a leading % CANNOT use a B-Tree index — causes a full table scan on every query. For text search on large tables, use FULLTEXT INDEX with MATCH AGAINST instead.

? What is the difference between LIKE and REGEXP?

✅LIKE uses only two wildcards (% and _) and is simpler/faster for basic patterns. REGEXP uses full regular expression syntax (^, $, [], {}, ., *, +, |) and is more powerful for complex patterns. LIKE can use an index when the pattern starts with a literal (no leading %); REGEXP generally cannot use a standard index.

17. IN Operator

Interview-Ready Definition

IN checks if a value matches ANY item in a static list or subquery result. It is cleaner and more readable than multiple OR conditions. NOT IN checks that a value does NOT appear in the list — but beware: NOT IN with NULLs in the subquery returns empty results!

Example 1: IN with Static List

courses table

course_id	title	level
1	Python for Data Science	Beginner
2	Machine Learning A-Z	Advanced
3	React JS Complete Guide	Intermediate
4	UI/UX Design Fundamentals	Beginner
5	MySQL Mastery	Intermediate
6	Flutter Mobile Dev	Advanced

```
-- Equivalent: WHERE level = 'Beginner' OR level = 'Advanced'
SELECT title, level, price
FROM courses
WHERE level IN ('Beginner', 'Advanced')
ORDER BY level, price;
```

Output:

```
title                      | level    | price
---------------------------+----------+---------
Deep Learning              | Advanced | 3499.00
Machine Learning A-Z       | Advanced | 2999.00
Flutter Mobile Dev         | Advanced | 2499.00
Advanced React Patterns    | Advanced | 2299.00
Python for Data Science    | Beginner | 1499.00
UI/UX Design Fundamentals  | Beginner | 999.00
Excel for Business         | Beginner | 699.00
```

How it works: IN is shorthand for multiple OR conditions. More readable with long lists — compare: WHERE city IN ('Delhi','Mumbai','Bangalore','Chennai') vs four separate OR conditions.

Example 2: IN with Subquery

```
-- Find students who have placed at least one completed order
SELECT full_name, email, city
FROM students
WHERE student_id IN (
    SELECT DISTINCT student_id
    FROM orders
    WHERE status = 'completed'
)
ORDER BY full_name;
```

Output:

```
full_name      | email                | city
```

```
--------------+--------------------+-----------
Amit Joshi    | amit@gmail.com     | Hyderabad
Anjali Singh  | anjali@gmail.com   | Mumbai
Deepak Verma  | deepak@gmail.com   | Pune
Kavya Reddy   | kavya@gmail.com    | Hyderabad
Meera Nair    | meera@gmail.com    | Kochi
Pooja Iyer    | pooja@gmail.com    | Chennai
Rahul Gupta   | rahul@gmail.com    | Delhi
Rohit Sharma  | rohit@gmail.com    | Bangalore
Sanjay Patel  | sanjay@gmail.com   | Ahmedabad
```

How it works: The subquery runs once and returns a set of student_ids. The outer query checks each student against that set. Nisha Agarwal (only has pending order) is excluded.

Example 3: NOT IN — Static List

```
-- Find students NOT from Hyderabad, Delhi, or Mumbai
SELECT full_name, city
FROM students
WHERE city NOT IN ('Hyderabad', 'Delhi', 'Mumbai')
ORDER BY city;
```

Output:

```
full_name     | city
--------------+-----------
Sanjay Patel  | Ahmedabad
Rohit Sharma  | Bangalore
Pooja Iyer    | Chennai
Nisha Agarwal | Jaipur
Meera Nair    | Kochi
Deepak Verma  | Pune
```

Example 4: NOT IN with Subquery — NULL Trap (Critical!)

```
-- DANGEROUS: If ANY student_id in the orders subquery is NULL,
-- the ENTIRE NOT IN returns EMPTY RESULT!
SELECT full_name FROM students
WHERE student_id NOT IN (
    SELECT student_id FROM orders   -- If student_id is ever NULL here → EMPTY RESULT!
);

-- SAFE ALTERNATIVE: NOT EXISTS (always works correctly with NULLs)
SELECT s.full_name FROM students s
WHERE NOT EXISTS (
    SELECT 1 FROM orders o WHERE o.student_id = s.student_id
);

-- SAFE ALTERNATIVE: LEFT JOIN + NULL check
SELECT s.full_name
FROM students s
LEFT JOIN orders o ON s.student_id = o.student_id
WHERE o.order_id IS NULL;
```

How it works: NOT IN with a subquery: if the subquery returns (1, 2, 3, NULL), then student_id NOT IN (..., NULL) = student_id != NULL = NULL (never TRUE) for every row — result is empty! NOT EXISTS handles NULLs correctly.

Pitfall: NEVER use NOT IN with a subquery on a column that might contain NULLs. Use NOT EXISTS or LEFT JOIN + IS NULL instead. This is the most dangerous SQL trap for beginners.

18. BETWEEN Operator

Interview-Ready Definition

BETWEEN low AND high tests an INCLUSIVE range — equivalent to col >= low AND col <= high. Works on numbers, dates, strings. NOT BETWEEN is the inverse. ALWAYS include both endpoints explicitly.

Example 1: BETWEEN with Numbers

courses — price column

title	price
Excel	699
UI/UX	999
MySQL	1299
Python	1499
Business	1799
React	1999
React Adv	2299
Flutter	2499
ML A-Z	2999
Deep Learning	3499

```
SELECT title, price
FROM courses
WHERE price BETWEEN 1000 AND 2000
ORDER BY price;
```

Output:

```
title                    | price
-------------------------+---------
MySQL Mastery            | 1299.00
Python for Data Science  | 1499.00
Business Analytics       | 1799.00
React JS Complete Guide  | 1999.00
```

How it works: BETWEEN 1000 AND 2000 INCLUDES both 1000 and 2000. Equivalent to: price >= 1000 AND price <= 2000. Excel (699) and UI/UX (999) are excluded; all courses 1000–2000 are included.

Example 2: BETWEEN with Dates — Orders in Q1 2022

orders table

order_id	order_date	amount_paid
1	2022-02-10 10:00:00	1499.00
2	2022-04-05 11:30:00	1799.00
5	2021-12-20 14:00:00	1499.00

order_id	order_date	amount_paid
6	2022-01-05 08:00:00	999.00

```
-- Q1 2022 = January, February, March
SELECT order_id, order_date, amount_paid
FROM orders
WHERE order_date BETWEEN '2022-01-01' AND '2022-03-31 23:59:59'
ORDER BY order_date;

-- Safer alternative (avoids end-time ambiguity):
SELECT order_id, order_date, amount_paid
FROM orders
WHERE order_date >= '2022-01-01'
  AND order_date <  '2022-04-01'   -- exclusive upper bound
ORDER BY order_date;
```

Output:

```
order_id | order_date          | amount_paid
---------+---------------------+------------
6        | 2022-01-05 08:00:00 | 999.00
1        | 2022-02-10 10:00:00 | 1499.00
```

How it works: For DATETIME columns, include 23:59:59 in the upper bound OR use < next_day. Orders from April (order 2) and December 2021 (order 5) are excluded. Q2 order (April 5) falls outside the range.

Pitfall: BETWEEN '2022-01-01' AND '2022-03-31' on a DATETIME column MISSES rows on March 31 after midnight (e.g., '2022-03-31 14:00:00') because DATETIME comparison treats '2022-03-31' as '2022-03-31 00:00:00'. Always add 23:59:59 or use < next day.

Example 3: NOT BETWEEN — Outside the Range

```
SELECT title, price
FROM courses
WHERE price NOT BETWEEN 1000 AND 2000
ORDER BY price;
```

Output:

```
title                      | price
---------------------------+---------
Excel for Business         | 699.00
UI/UX Design Fundamentals  | 999.00
Advanced React Patterns    | 2299.00
Flutter Mobile Dev         | 2499.00
Machine Learning A-Z       | 2999.00
Deep Learning              | 3499.00
```

How it works: NOT BETWEEN 1000 AND 2000 is equivalent to price < 1000 OR price > 2000. The endpoints 1000 and 2000 are EXCLUDED from NOT BETWEEN results.

Example 4: BETWEEN with Strings — Alphabetical Range

```
-- Students whose name starts with letters B through M
SELECT full_name FROM students
WHERE full_name BETWEEN 'B' AND 'N'
ORDER BY full_name;
```

Output:

```
full_name
-----------
Deepak Verma
Kavya Reddy
```

```
Meera Nair
```

 How it works: String BETWEEN uses alphabetical/lexicographic ordering. Names starting with D, K, M fall between B and N. Anjali (A) is excluded; Nisha starts with N but BETWEEN is exclusive at N (N...) check).

19. Aliases (AS)

Interview-Ready Definition

An alias gives a column or table a temporary name for the query duration. Column aliases improve result readability. Table aliases shorten JOIN syntax and are REQUIRED for self-joins. The AS keyword is optional but recommended for clarity.

Example 1: Column Aliases with Expressions

```
SELECT
  title                              AS course_name,
  price                              AS price_inr,
  ROUND(price * 1.18, 2)             AS price_with_gst,
  CONCAT(duration_hrs, ' hrs')       AS duration,
  CASE level
    WHEN 'Beginner' THEN 'Entry'
    WHEN 'Intermediate' THEN 'Mid'
    WHEN 'Advanced' THEN 'Expert'
  END                                AS level_short
FROM courses
ORDER BY price_inr DESC
LIMIT 4;
```

Output:

```
course_name          | price_inr | price_with_gst | duration | level_short
---------------------+-----------+----------------+----------+------------
Deep Learning        | 3499.00   | 4128.82        | 70 hrs   | Expert
Machine Learning A-Z | 2999.00   | 3538.82        | 60 hrs   | Expert
Flutter Mobile Dev   | 2499.00   | 2948.82        | 50 hrs   | Expert
Advanced React Patt. | 2299.00   | 2712.82        | 28 hrs   | Expert
```

How it works: AS renames the column in the output. Aliases can be used in ORDER BY. They CANNOT be used in WHERE (aliases don't exist at WHERE evaluation time).

Example 2: Table Aliases — Cleaner Multi-Table Queries

```
-- Without aliases (verbose and hard to read):
SELECT students.full_name, courses.title, orders.amount_paid
FROM orders
JOIN students ON orders.student_id = students.student_id
JOIN courses  ON orders.course_id  = courses.course_id
LIMIT 3;

-- With aliases (clean and concise):
SELECT s.full_name, c.title, o.amount_paid, o.status
FROM orders o                -- 'o' = orders
JOIN students s ON o.student_id = s.student_id   -- 's' = students
JOIN courses  c ON o.course_id  = c.course_id    -- 'c' = courses
WHERE o.status = 'completed'
ORDER BY s.full_name
LIMIT 5;
```

Output:

```
full_name    | title                    | amount_paid | status
-------------+--------------------------+-------------+-----------
Amit Joshi   | Python for Data Science  | 1499.00     | completed
Amit Joshi   | React JS Complete Guide  | 1799.00     | completed
Anjali Singh | Flutter Mobile Dev       | 2499.00     | completed
Deepak Verma | React JS Complete Guide  | 1999.00     | completed
```

```
Kavya Reddy  | Python for Data Science | 1499.00     | completed
```

 How it works: Table aliases eliminate table-name repetition. Once you write FROM orders o, use o everywhere instead of orders. Required when the same table appears twice (self-join).

Example 3: Alias Without AS Keyword

```
-- AS keyword is optional (both syntaxes work):
SELECT title AS course_name, price AS cost FROM courses;
SELECT title    course_name, price    cost  FROM courses;  -- same result

-- Table alias without AS:
SELECT o.order_id, s.full_name
FROM orders o JOIN students s ON o.student_id = s.student_id LIMIT 3;
```

 How it works: Omitting AS works for both column and table aliases. However, using AS makes the code more readable — especially for reviewers who are not familiar with the shorthand.

20. MySQL Operators — Full Reference

Interview-Ready Definition

MySQL operators perform operations on values: arithmetic (math), comparison (true/false tests), logical (AND/OR/NOT), bitwise (bit-level), and assignment. Operator precedence determines evaluation order — use parentheses to make intent explicit.

Arithmetic Operators

Operator	Name	Example	Result
+	Addition	price + 100	price increased by 100
-	Subtraction	price - discount	price after discount
*	Multiplication	price * 1.18	18% GST applied
/	Division	price / duration_hrs	Cost per hour
%	Modulo	10 % 3	1 (remainder)
DIV	Integer division	10 DIV 3	3 (whole number only)

Example 1: Arithmetic in SELECT

```
SELECT
  title,
  price,
  ROUND(price * 0.18, 2)           AS gst_amount,
  ROUND(price * 1.18, 2)           AS total_with_gst,
  ROUND(price * 0.90, 2)           AS after_10pct_discount,
  ROUND(price / duration_hrs, 2)   AS cost_per_hour,
  price % 100                      AS cents_portion
FROM courses
WHERE level = 'Advanced'
ORDER BY price DESC;
```

Output:

```
title                    | price    | gst_amount | total_with_gst | after_discount | cost/hr |
cents
-------------------------+----------+------------+----------------+----------------+---------+-----
-
Deep Learning            | 3499.00 | 629.82     | 4128.82        | 3149.10        | 49.99   | 99
Machine Learning A-Z     | 2999.00 | 539.82     | 3538.82        | 2699.10        | 49.98   | 99
Flutter Mobile Dev       | 2499.00 | 449.82     | 2948.82        | 2249.10        | 49.98   | 99
Advanced React Patterns  | 2299.00 | 413.82     | 2712.82        | 2069.10        | 82.11   | 99
```

Comparison Operators (Full Reference)

Operator	Meaning	NULL-Safe?	Example
=	Equal	No (NULL=NULL → NULL)	level = 'Beginner'
<>	Not equal	No	status <> 'refunded'
!=	Not equal (same)	No	status != 'refunded'

Operator	Meaning	NULL-Safe?	Example
>	Greater than	No	price > 2000
<	Less than	No	rating < 4
>=	Greater or equal	No	duration_hrs >= 30
<=	Less or equal	No	price <= 1000
<=>	NULL-safe equal	YES!	phone <=> NULL (same as IS NULL but allows comparison)

Logical Operators

Operator	Precedence	Meaning
NOT	Highest	Negates condition
AND	Middle	Both conditions must be TRUE
OR	Lowest	At least one condition must be TRUE
XOR	Like OR	Exactly one (not both) must be TRUE

Example 2: Logical Operators with NULL

```
SELECT
  NULL AND TRUE   AS null_and_true,    -- NULL
  NULL AND FALSE  AS null_and_false,   -- FALSE (FALSE wins in AND)
  NULL OR  TRUE   AS null_or_true,     -- TRUE  (TRUE wins in OR)
  NULL OR  FALSE  AS null_or_false,    -- NULL
  NOT NULL        AS not_null;         -- NULL
```

Output:

```
null_and_true | null_and_false | null_or_true | null_or_false | not_null
--------------+----------------+--------------+---------------+---------
NULL          | FALSE          | TRUE         | NULL          | NULL
```

Operator Precedence (Highest to Lowest)

Priority	Operators
1 (Highest)	INTERVAL, BINARY, COLLATE
2	! (NOT bit)
3	- (unary), ~ (bit NOT)
4	^ (bit XOR)
5	* / DIV % MOD
6	+ -
7	<< >> (bit shift)
8	& (bit AND)
9	| (bit OR)

Priority	Operators
10	= <=> < > <= >= <> != IS LIKE REGEXP IN BETWEEN
11	NOT
12	AND &&
13	XOR
14 (Lowest)	OR ||

 Tip: When in doubt, use parentheses. They cost nothing in performance and prevent subtle bugs from unexpected precedence.

21. INNER JOIN

⍰ Interview-Ready Definition

INNER JOIN returns ONLY rows where the join condition matches in BOTH tables. Rows that have no matching record in either table are excluded. Most common join type. "INNER" keyword is optional — plain JOIN means INNER JOIN.

▶ students table (left)

student_id	full_name
1	Amit Joshi
2	Meera Nair
3	Rahul Gupta
11	New Student — no orders

▶ orders table (right)

order_id	student_id	course_id
1	1	1
2	1	3
3	2	2
99	999	1 — student 999 does not exist

Example 1: Basic INNER JOIN — Students and Their Orders

```
SELECT
  s.full_name,
  c.title          AS course,
  o.amount_paid,
  o.status,
  o.order_date
FROM orders o
INNER JOIN students s ON o.student_id = s.student_id
INNER JOIN courses  c ON o.course_id  = c.course_id
WHERE o.status = 'completed'
ORDER BY s.full_name, o.order_date;
```

⍰ Output:

```
full_name     | course                    | amount_paid | status    | order_date
--------------+---------------------------+-------------+-----------+--------------------
Amit Joshi    | Python for Data Science   | 1499.00     | completed | 2022-02-10 10:00:00
Amit Joshi    | React JS Complete Guide   | 1799.00     | completed | 2022-04-05 11:30:00
Anjali Singh  | Flutter Mobile Dev        | 2499.00     | completed | 2023-03-10 10:00:00
Deepak Verma  | React JS Complete Guide   | 1999.00     | completed | 2021-09-10 10:00:00
Kavya Reddy   | Python for Data Science   | 1499.00     | completed | 2023-01-15 11:00:00
Kavya Reddy   | Machine Learning A-Z      | 2999.00     | completed | 2023-02-20 13:00:00
Meera Nair    | Machine Learning A-Z      | 2999.00     | completed | 2022-05-01 09:00:00
Meera Nair    | MySQL Mastery             | 1299.00     | completed | 2023-02-14 15:00:00
Pooja Iyer    | Advanced React Patterns   | 2299.00     | completed | 2023-06-01 12:00:00
Rahul Gupta   | Python for Data Science   | 1499.00     | completed | 2021-12-20 14:00:00
Rohit Sharma  | Deep Learning             | 3499.00     | completed | 2021-01-15 09:00:00
Sanjay Patel  | Business Analytics        | 1799.00     | completed | 2022-08-12 16:00:00
```

```
Sanjay Patel  | Excel for Business       | 699.00      | completed | 2022-09-01 09:30:00
-- Note: 'New Student' (id=11) is NOT here — no orders exist for them
```

How it works: Student 11 (New Student) has no orders → excluded. Order 99 references student_id 999 → student 999 doesn't exist → excluded. INNER JOIN is strict: both sides must match.

Example 2: Three-Table INNER JOIN with Aggregation

```
-- Revenue by course category
SELECT
  cat.category_name,
  COUNT(o.order_id)             AS total_orders,
  SUM(o.amount_paid)            AS total_revenue,
  ROUND(AVG(o.amount_paid), 2)  AS avg_order_value
FROM orders o
INNER JOIN courses    c   ON o.course_id   = c.course_id
INNER JOIN categories cat ON c.category_id = cat.category_id
WHERE o.status = 'completed'
GROUP BY cat.category_id, cat.category_name
ORDER BY total_revenue DESC;
```

Output:

```
category_name      | total_orders | total_revenue | avg_order_value
-------------------+--------------+---------------+----------------
Data Science       | 6            | 13993.00      | 2332.17
Web Development    | 5            | 8795.00       | 1759.00
Mobile Development | 1            | 2499.00       | 2499.00
Business           | 2            | 2498.00       | 1249.00
Design             | 1            | 999.00        | 999.00
```

How it works: Three-table chain: orders → courses → categories. GROUP BY aggregates by category. Data Science generates the most revenue.

22. LEFT JOIN

⍰ Interview-Ready Definition

LEFT JOIN returns ALL rows from the LEFT table, plus matching rows from the RIGHT table. When there is no match, right-side columns contain NULL. Use LEFT JOIN when you want to keep all records from the primary (left) table regardless of whether related records exist.

▶ students (LEFT table) — all 10 rows

student_id	full_name
1	Amit Joshi (has orders)
2	Meera Nair (has orders)
11	No-Order Student (no orders in orders table)

Example 1: LEFT JOIN — All Students with Order Counts

```
SELECT
  s.full_name,
  COUNT(o.order_id)              AS total_orders,
  IFNULL(SUM(o.amount_paid), 0)  AS total_spent
FROM students s
LEFT JOIN orders o ON s.student_id = o.student_id
GROUP BY s.student_id, s.full_name
ORDER BY total_spent DESC;
```

⍰ Output:

```
full_name     | total_orders | total_spent
--------------+--------------+------------
Kavya Reddy   | 2            | 4498.00
Meera Nair    | 2            | 4298.00
Rohit Sharma  | 1            | 3499.00
Amit Joshi    | 2            | 3298.00
Anjali Singh  | 1            | 2499.00
Sanjay Patel  | 2            | 2498.00
Pooja Iyer    | 1            | 2299.00
Deepak Verma  | 1            | 1999.00
Rahul Gupta   | 2            | 2498.00
Nisha Agarwal | 1            | 1299.00  ← pending only, still counted
(All 10 students appear — even those with only pending orders)
```

⍰ How it works: LEFT JOIN keeps ALL students. o.order_id is NULL for students with no matching order → COUNT(o.order_id) = 0. IFNULL(SUM(...), 0) converts NULL sum to 0.

Example 2: LEFT JOIN — Find Courses With No Reviews (NULL Pattern)

```
SELECT
  c.title,
  c.price,
  COUNT(r.review_id)  AS review_count
FROM courses c
LEFT JOIN reviews r ON c.course_id = r.course_id
GROUP BY c.course_id, c.title, c.price
HAVING COUNT(r.review_id) = 0   -- Only courses with NO reviews
ORDER BY c.price DESC;
```

⍰ Output:

```
title                       | price   | review_count
---------------------------+---------+-------------
Flutter Mobile Dev          | 2499.00 | 0
Business Analytics          | 1799.00 | 0
MySQL Mastery               | 1299.00 | 0
UI/UX Design Fundamentals   | 999.00  | 0
```

How it works: LEFT JOIN brings all courses. Courses with no reviews have r.review_id = NULL. COUNT(r.review_id) = 0 for these (COUNT skips NULLs). HAVING filters to show only these courses.

Tip: LEFT JOIN + WHERE right_side IS NULL and LEFT JOIN + HAVING COUNT = 0 are the two standard patterns for finding orphan records.

Example 3: LEFT JOIN — Filter NULL rows (Anti-Join Pattern)

```
-- Find students who have NEVER placed any order
SELECT s.full_name, s.email
FROM students s
LEFT JOIN orders o ON s.student_id = o.student_id
WHERE o.order_id IS NULL   -- NULL = no matching order found
ORDER BY s.full_name;
```

Output:

```
(All students in sample data have at least one order, so result is empty in this dataset)

If a student had no orders, they would appear here with o.order_id = NULL
```

How it works: LEFT JOIN brings all students. WHERE o.order_id IS NULL filters to only those with no matching order — the classic anti-join pattern.

23. RIGHT JOIN

⍰ Interview-Ready Definition

RIGHT JOIN returns ALL rows from the RIGHT table, plus matching rows from the LEFT table. It is the mirror image of LEFT JOIN. Most developers rewrite RIGHT JOINs as LEFT JOINs by swapping table order — both give identical results.

Example 1: RIGHT JOIN — All Instructors Including Those With No Courses

▶ instructors table (RIGHT side)

instructor_id	full_name	rating
1	Ravi Kumar	4.80
2	Priya Sharma	4.60
3	Arjun Mehta	4.90
4	Sneha Reddy	4.30
5	Vikram Singh	NULL — no courses yet

```
SELECT
  i.full_name       AS instructor,
  COUNT(c.course_id) AS course_count,
  ROUND(AVG(c.price), 2) AS avg_course_price
FROM courses c
RIGHT JOIN instructors i ON c.instructor_id = i.instructor_id
GROUP BY i.instructor_id, i.full_name
ORDER BY course_count DESC;
```

⍰ Output:

```
instructor    | course_count | avg_course_price
--------------+--------------+-----------------
Arjun Mehta   | 3            | 2999.00
Priya Sharma  | 3            | 2032.33
Ravi Kumar    | 2            | 1399.00
Sneha Reddy   | 2            | 849.00
Vikram Singh  | 0            | NULL   ← No courses, but still appears!
```

⍰ **How it works:** courses is on the LEFT, instructors on the RIGHT. RIGHT JOIN guarantees all instructors appear. Vikram (no courses) gets count=0 and avg=NULL.

Example 2: RIGHT JOIN Rewritten as LEFT JOIN (Identical Result)

```
-- These two queries produce IDENTICAL results:

-- RIGHT JOIN version:
SELECT i.full_name, COUNT(c.course_id) AS cnt
FROM courses c
RIGHT JOIN instructors i ON c.instructor_id = i.instructor_id
GROUP BY i.instructor_id, i.full_name;

-- LEFT JOIN version (same logic, just swap table order):
SELECT i.full_name, COUNT(c.course_id) AS cnt
FROM instructors i        -- instructors is now the LEFT table
LEFT JOIN courses c ON i.instructor_id = c.instructor_id
```

```
GROUP BY i.instructor_id, i.full_name;
```

How it works: Any RIGHT JOIN can be rewritten as a LEFT JOIN by reversing the table order. Most teams standardize on LEFT JOIN for consistency. The query optimizer produces the same plan for both.

24. CROSS JOIN

⍰ Interview-Ready Definition

CROSS JOIN produces the Cartesian product — every row from Table A paired with every row from Table B. Result rows = |A| × |B|. No ON condition needed or allowed. Use for generating all possible combinations in reports, test data, or schedule grids.

▶ levels (3 rows)

level
Beginner
Intermediate
Advanced

▶ categories (5 rows)

category_name
Data Science
Web Development
Design
Business
Mobile Development

Example 1: CROSS JOIN — Generate All Level × Category Combinations

```
-- Using a virtual table with UNION for levels
SELECT
  lev.level,
  cat.category_name,
  COUNT(c.course_id) AS courses_available
FROM (
  SELECT 'Beginner'     AS level
  UNION ALL SELECT 'Intermediate'
  UNION ALL SELECT 'Advanced'
) lev
CROSS JOIN categories cat
LEFT JOIN courses c
  ON c.level = lev.level AND c.category_id = cat.category_id
GROUP BY lev.level, cat.category_name
ORDER BY lev.level, cat.category_name;
```

⍰ Output:

```
level         | category_name       | courses_available
-------------+--------------------+------------------
Advanced      | Business            | 0   ← gap in curriculum!
Advanced      | Data Science        | 2
Advanced      | Design              | 0
Advanced      | Mobile Development  | 1
Advanced      | Web Development     | 1
Beginner      | Business            | 1
Beginner      | Data Science        | 1
Beginner      | Design              | 1
```

```
Beginner     | Mobile Development | 0
Beginner     | Web Development    | 0
Intermediate | Business           | 1
Intermediate | Data Science       | 0
Intermediate | Design             | 0
Intermediate | Mobile Development | 0
Intermediate | Web Development    | 2
(15 rows = 3 levels × 5 categories)
```

 How it works: CROSS JOIN generates the "skeleton" of all 15 possible slots. LEFT JOIN fills in actual course counts. Rows with 0 reveal curriculum gaps — no Advanced Business course exists.

 Pitfall: CROSS JOIN on large tables is dangerous. 1,000 rows × 1,000 rows = 1,000,000 rows returned. Always restrict at least one table to a small reference set.

25. SELF JOIN

Interview-Ready Definition

A SELF JOIN joins a table to ITSELF using two different aliases. Used to compare rows within the same table — finding related records, hierarchies (employee → manager), duplicate pairs, or courses in the same category.

Example 1: SELF JOIN — Find Courses in the Same Category

courses table (subset)

course_id	title	category_id
1	Python for Data Science	1
2	Machine Learning A-Z	1
3	React JS Complete Guide	2
5	MySQL Mastery	2
10	Deep Learning	1

```
SELECT
  a.title      AS course_1,
  b.title      AS course_2,
  a.level      AS level_1,
  b.level      AS level_2
FROM courses a
JOIN courses b
  ON a.category_id = b.category_id     -- same category
 AND a.course_id   < b.course_id       -- prevent (A,B) AND (B,A) duplicates
ORDER BY a.category_id, a.course_id;
```

Output:

```
course_1                  | course_2                  | level_1    | level_2
--------------------------+---------------------------+------------+----------
Python for Data Science   | Machine Learning A-Z      | Beginner   | Advanced
Python for Data Science   | Deep Learning             | Beginner   | Advanced
Machine Learning A-Z      | Deep Learning             | Advanced   | Advanced
React JS Complete Guide   | MySQL Mastery             | Intermed   | Intermed
React JS Complete Guide   | Advanced React Patterns   | Intermed   | Advanced
MySQL Mastery             | Advanced React Patterns   | Intermed   | Advanced
Business Analytics        | Excel for Business        | Intermed   | Beginner
```

How it works: Alias a is the first course, alias b is the second copy. a.course_id < b.course_id prevents showing both (Python, ML) and (ML, Python) and prevents self-pairing (Python, Python).

Example 2: SELF JOIN — Students in the Same City

```
SELECT
  a.full_name AS student_1,
  b.full_name AS student_2,
  a.city
FROM students a
JOIN students b
  ON a.city       = b.city
 AND a.student_id < b.student_id
ORDER BY a.city;
```

Output:

```
student_1     | student_2    | city
-------------+-------------+-----------
Amit Joshi    | Kavya Reddy | Hyderabad
```

How it works: Only Amit and Kavya share a city (Hyderabad). The a.student_id < b.student_id condition ensures each pair appears only once.

26. UNION & UNION ALL

Interview-Ready Definition

UNION combines results of two SELECT statements. Rules: (1) Same number of columns in both SELECTs. (2) Corresponding columns must have compatible types. UNION removes duplicate rows (slower). UNION ALL keeps ALL rows including duplicates (faster).

Example 1: UNION — Combine Without Duplicates

students.city (before UNION)

city
Hyderabad
Kochi
Delhi
Hyderabad (dup!)

instructors.city

city
Hyderabad (another dup!)
Bangalore
Mumbai
Chennai
Delhi (dup!)

```
-- UNION: removes ALL duplicates across both result sets
SELECT city FROM students
UNION
SELECT city FROM instructors
ORDER BY city;
```

Output:

```
city
-----------
Ahmedabad
Bangalore
Chennai
Delhi       ← appears in both tables, shown once
Hyderabad   ← appears multiple times, shown once
Jaipur
Kochi
Mumbai
Pune
```

How it works: UNION deduplicates across both result sets — Hyderabad from students and Hyderabad from instructors become one row. This is slower because MySQL must sort and compare all results.

Example 2: UNION ALL — Keep All Rows Including Duplicates

```
-- UNION ALL: keeps every row, including duplicates (faster)
SELECT city, 'Student'    AS source FROM students
UNION ALL
SELECT city, 'Instructor' AS source FROM instructors
ORDER BY city;
```

Output:

```
city        | source
------------+------------
Ahmedabad   | Student
Bangalore   | Student
Bangalore   | Instructor  ← Bangalore appears twice
Chennai     | Student
Chennai     | Instructor  ← Chennai appears twice
Delhi       | Student
Delhi       | Instructor  ← Delhi appears twice
Hyderabad   | Student
Hyderabad   | Student
Hyderabad   | Instructor  ← THREE Hyderabad rows
...
```

How it works: UNION ALL appends result sets without any deduplication — fastest option. Adding a 'source' column shows which table each row came from.

Example 3: UNION to Build a Combined Directory

```
-- Create a unified "People" directory from two tables
SELECT
  full_name,
  email,
  'Student'    AS role,
  city
FROM students
UNION ALL
SELECT
  full_name,
  email,
  'Instructor' AS role,
  city
FROM instructors
ORDER BY role, full_name;
```

Output:

```
full_name     | email                | role       | city
--------------+----------------------+------------+-----------
Arjun Mehta   | arjun@edushop.com    | Instructor | Mumbai
Priya Sharma  | priya@edushop.com    | Instructor | Bangalore
Ravi Kumar    | ravi@edushop.com     | Instructor | Hyderabad
Sneha Reddy   | sneha@edushop.com    | Instructor | Chennai
Vikram Singh  | vikram@edushop.com   | Instructor | Delhi
Amit Joshi    | amit@gmail.com       | Student    | Hyderabad
Anjali Singh  | anjali@gmail.com     | Student    | Mumbai
...
```

How it works: Both SELECT queries have 4 columns with matching types. UNION ALL combines them. A literal string 'Student' / 'Instructor' creates a discriminator column. ORDER BY applies to the combined result.

? What is the difference between UNION and UNION ALL?

✅UNION combines result sets AND removes duplicate rows — MySQL must sort and compare all rows to find duplicates, making it slower. UNION ALL simply appends result sets and keeps ALL rows including duplicates — no deduplication, so it is faster. Use UNION ALL when you know there are no duplicates or you want to keep them.

27. GROUP BY

Interview-Ready Definition

GROUP BY groups rows with matching values into summary rows, then applies aggregate functions to each group. Every column in SELECT must either appear in GROUP BY or be inside an aggregate function. Executed AFTER WHERE, BEFORE HAVING.

Example 1: GROUP BY — Orders per Status

▶ orders table — before grouping

order_id	student_id	amount_paid	status
1	1	1499	completed
2	1	1799	completed
3	2	2999	completed
6	3	999	refunded
13	8	1299	pending

```
SELECT
  status,
  COUNT(*)           AS order_count,
  SUM(amount_paid)   AS total_amount,
  MIN(amount_paid)   AS min_order,
  MAX(amount_paid)   AS max_order
FROM orders
GROUP BY status
ORDER BY order_count DESC;
```

Output:

```
status    | order_count | total_amount | min_order | max_order
----------+-------------+--------------+-----------+----------
completed | 13          | 26488.00     | 699.00    | 3499.00
pending   | 1           | 1299.00      | 1299.00   | 1299.00
refunded  | 1           | 999.00       | 999.00    | 999.00
```

How it works: GROUP BY status creates one result row per unique status value. All aggregate functions (COUNT, SUM, MIN, MAX) operate within each group independently.

Example 2: GROUP BY Multiple Columns

```
SELECT
  c.level,
  cat.category_name,
  COUNT(o.order_id)  AS orders,
  SUM(o.amount_paid) AS revenue
FROM orders o
JOIN courses    c   ON o.course_id    = c.course_id
JOIN categories cat ON c.category_id = cat.category_id
WHERE o.status = 'completed'
GROUP BY c.level, cat.category_name
ORDER BY c.level, revenue DESC;
```

Output:

```
level        | category_name      | orders | revenue
```

```
-------------+-------------------+--------+---------
Advanced     | Data Science      | 3      | 8997.00
Advanced     | Mobile Dev        | 1      | 2499.00
Advanced     | Web Development   | 1      | 2299.00
Beginner     | Data Science      | 3      | 4497.00
Beginner     | Business          | 1      | 699.00
Intermediate | Web Development   | 4      | 6496.00
Intermediate | Business          | 1      | 1799.00
```

Example 3: GROUP BY with ROLLUP — Subtotals and Grand Total

```
SELECT
  IFNULL(level, '★ GRAND TOTAL ★') AS level,
  COUNT(*)                         AS course_count,
  SUM(price)                       AS total_price,
  ROUND(AVG(price), 2)             AS avg_price
FROM courses
GROUP BY level WITH ROLLUP;
```

Output:

```
level               | course_count | total_price | avg_price
-------------------+--------------+-------------+-----------
Advanced            | 4            | 11296.00    | 2824.00
Beginner            | 3            | 3197.00     | 1065.67
Intermediate        | 3            | 5097.00     | 1699.00
★ GRAND TOTAL ★    | 10           | 19590.00    | 1959.00
```

How it works: WITH ROLLUP adds an extra row with overall aggregates. The grand total row has NULL for the grouped column — IFNULL replaces it with a label.

28. HAVING Clause

Interview-Ready Definition

HAVING filters GROUPS after GROUP BY — like WHERE but for aggregated results. WHERE filters individual rows (before grouping). HAVING filters group summaries (after grouping). HAVING can use aggregate functions; WHERE cannot.

WHERE vs HAVING — Side-by-Side

Feature	WHERE	HAVING
When it runs	BEFORE GROUP BY	AFTER GROUP BY
Filters	Individual rows	Groups / aggregated results
Can use aggregates?	No (SUM, COUNT etc.)	Yes (SUM, COUNT, AVG, MIN, MAX)
Can use aliases?	No (too early)	Yes (in MySQL)
Works without GROUP BY?	Yes	Yes (filters ungrouped result as one group)

Example 1: HAVING — Filter Groups by Aggregate

```
-- Students who spent over ₹3000 total (completed orders only)
SELECT
  s.full_name,
  COUNT(o.order_id)   AS total_orders,
  SUM(o.amount_paid)  AS total_spent
FROM students s
JOIN orders o ON s.student_id = o.student_id
WHERE o.status = 'completed'          -- WHERE: filter rows before grouping
GROUP BY s.student_id, s.full_name   -- GROUP BY: create one row per student
HAVING SUM(o.amount_paid) > 3000     -- HAVING: filter groups by total
ORDER BY total_spent DESC;
```

Output:

```
full_name     | total_orders | total_spent
-------------+--------------+------------
Kavya Reddy   | 2            | 4498.00
Meera Nair    | 2            | 4298.00
Rohit Sharma  | 1            | 3499.00
Amit Joshi    | 2            | 3298.00
```

How it works: WHERE o.status='completed' filters rows before grouping. Then grouping happens. Then HAVING SUM(amount_paid) > 3000 filters out student groups with smaller totals. You CANNOT write WHERE SUM(amount_paid) > 3000.

Example 2: HAVING with COUNT

```
-- Courses that have received 2 or more reviews
SELECT
  c.title,
  COUNT(r.review_id)          AS review_count,
  ROUND(AVG(r.rating), 2)     AS avg_rating
FROM courses c
JOIN reviews r ON c.course_id = r.course_id
GROUP BY c.course_id, c.title
```

```
HAVING COUNT(r.review_id) >= 2
ORDER BY avg_rating DESC;
```

Output:

```
title                    | review_count | avg_rating
-------------------------+--------------+-----------
Machine Learning A-Z     | 2            | 5.00
Python for Data Science  | 3            | 4.67
```

Example 3: HAVING with Multiple Conditions

```
-- Instructors with 2+ courses AND average price over 1500
SELECT
  i.full_name,
  COUNT(c.course_id)          AS course_count,
  ROUND(AVG(c.price), 2)      AS avg_price
FROM instructors i
JOIN courses c ON i.instructor_id = c.instructor_id
GROUP BY i.instructor_id, i.full_name
HAVING COUNT(c.course_id) >= 2
   AND AVG(c.price) > 1500
ORDER BY avg_price DESC;
```

Output:

```
full_name      | course_count | avg_price
---------------+--------------+-----------
Arjun Mehta    | 3            | 2999.00
Priya Sharma   | 3            | 2032.33
Ravi Kumar     | 2            | 1399.00   ← excluded! avg 1399 < 1500
```

How it works: Ravi Kumar has 2 courses but average price 1399 — below 1500. HAVING uses AND to combine two aggregate conditions.

29. CASE Expression

Interview-Ready Definition

CASE is SQL's conditional expression — returns a value based on conditions, like an IF-ELSE. Two forms: Simple CASE (compare one column to values) and Searched CASE (evaluate conditions). Can be used in SELECT, WHERE, ORDER BY, GROUP BY, and inside aggregate functions.

Two CASE Syntax Forms

```
-- Form 1: Simple CASE (compare one value)
CASE column
  WHEN value1 THEN result1
  WHEN value2 THEN result2
  ELSE default_result
END

-- Form 2: Searched CASE (evaluate conditions)
CASE
  WHEN condition1 THEN result1
  WHEN condition2 THEN result2
  ELSE default_result
END
```

Example 1: Simple CASE — Label Translation

▶ courses — level column before

title	level
Python	Beginner
React JS	Intermediate
ML A-Z	Advanced

```
SELECT
  title,
  level,
  CASE level
    WHEN 'Beginner'     THEN '□ Entry Level — No Prerequisites'
    WHEN 'Intermediate' THEN '□ Mid Level — Basic Knowledge Required'
    WHEN 'Advanced'     THEN '□ Expert Level — Strong Foundation Needed'
    ELSE 'O Unknown Level'
  END AS level_description
FROM courses;
```

Output:

```
title                      | level        | level_description
---------------------------+--------------+-----------------------------------
Python for Data Science    | Beginner     | □ Entry Level — No Prerequisites
Machine Learning A-Z       | Advanced     | □ Expert Level — Strong Foundation
React JS Complete Guide    | Intermediate | □ Mid Level — Basic Knowledge
UI/UX Design Fundamentals  | Beginner     | □ Entry Level — No Prerequisites
...
```

Example 2: Searched CASE — Price Brackets

```
SELECT
  title,
  price,
  CASE
    WHEN price < 1000                THEN 'Budget   (< ₹1K)'
    WHEN price BETWEEN 1000 AND 1999 THEN 'Standard (₹1K-2K)'
    WHEN price BETWEEN 2000 AND 2999 THEN 'Premium  (₹2K-3K)'
    ELSE                                  'Elite    (₹3K+)'
  END AS price_bracket
FROM courses
ORDER BY price;
```

Output:

```
title                      | price    | price_bracket
---------------------------+----------+------------------
Excel for Business         | 699.00   | Budget   (< ₹1K)
UI/UX Design Fund.         | 999.00   | Budget   (< ₹1K)
MySQL Mastery              | 1299.00  | Standard (₹1K-2K)
Python for Data Science    | 1499.00  | Standard (₹1K-2K)
Business Analytics         | 1799.00  | Standard (₹1K-2K)
React JS Complete Guide    | 1999.00  | Standard (₹1K-2K)
Advanced React Patterns    | 2299.00  | Premium  (₹2K-3K)
Flutter Mobile Dev         | 2499.00  | Premium  (₹2K-3K)
Machine Learning A-Z       | 2999.00  | Premium  (₹2K-3K)
Deep Learning              | 3499.00  | Elite    (₹3K+)
```

Example 3: CASE in SUM — Conditional Pivot Table

```
-- Revenue split by course level in a single row (pivot-style)
SELECT
  SUM(CASE WHEN c.level = 'Beginner'     THEN o.amount_paid ELSE 0 END) AS beginner_rev,
  SUM(CASE WHEN c.level = 'Intermediate' THEN o.amount_paid ELSE 0 END) AS
intermediate_rev,
  SUM(CASE WHEN c.level = 'Advanced'     THEN o.amount_paid ELSE 0 END) AS advanced_rev,
  SUM(o.amount_paid)                                                     AS grand_total
FROM orders o
JOIN courses c ON o.course_id = c.course_id
WHERE o.status = 'completed';
```

Output:

```
beginner_rev | intermediate_rev | advanced_rev | grand_total
-------------+------------------+--------------+------------
5196.00      | 6496.00          | 14796.00     | 26488.00
```

How it works: CASE inside SUM() creates conditional aggregation — no GROUP BY needed! This is the SQL pivot pattern: each CASE column shows revenue for one category.

Example 4: CASE in ORDER BY — Custom Sort Priority

```
-- Show courses in order: Active > Inactive, then by price
SELECT title, is_active, price
FROM courses
ORDER BY
  CASE is_active WHEN 1 THEN 0 ELSE 1 END,  -- active courses first
  price DESC;
```

How it works: CASE returns 0 for active courses and 1 for inactive — ordering them first. The secondary sort is price descending within each group.

30. EXISTS Operator

Interview-Ready Definition

EXISTS returns TRUE if the subquery returns at least one row, FALSE otherwise. It short-circuits — stops searching the subquery as soon as one row is found. More efficient than IN for large datasets. Correlated with the outer query.

Example 1: EXISTS — Students Who Have Placed Orders

```
SELECT full_name, email
FROM students s
WHERE EXISTS (
    SELECT 1        -- SELECT 1 is convention; the value doesn't matter
    FROM orders o
    WHERE o.student_id = s.student_id    -- correlated: references outer 's'
      AND o.status = 'completed'
)
ORDER BY full_name;
```

Output:

```
full_name     | email
--------------+-------------------
Amit Joshi    | amit@gmail.com
Anjali Singh  | anjali@gmail.com
Deepak Verma  | deepak@gmail.com
Kavya Reddy   | kavya@gmail.com
Meera Nair    | meera@gmail.com
Pooja Iyer    | pooja@gmail.com
Rahul Gupta   | rahul@gmail.com
Rohit Sharma  | rohit@gmail.com
Sanjay Patel  | sanjay@gmail.com
```

How it works: EXISTS asks: "does any completed order exist for this student?" The SELECT 1 value is irrelevant — only WHETHER any row is returned matters. As soon as one order is found, EXISTS returns TRUE and stops searching.

Example 2: NOT EXISTS — Safe Alternative to NOT IN

```
-- Courses that have NEVER been ordered
SELECT c.title, c.price
FROM courses c
WHERE NOT EXISTS (
    SELECT 1 FROM orders o WHERE o.course_id = c.course_id
);

-- Compare to NOT IN (DANGEROUS if NULLs exist):
SELECT title FROM courses
WHERE course_id NOT IN (SELECT course_id FROM orders);
-- If any course_id in orders is NULL → returns EMPTY result (wrong!)
```

Output:

```
-- NOT EXISTS (safe result — courses never ordered):
title | price
(In sample data, all courses have at least one order)
(If a course had no orders, it would appear here)
```

How it works: NOT EXISTS is NULL-safe: works correctly regardless of NULLs. NOT IN with a subquery that returns NULLs fails silently by returning no rows.

31. ANY and ALL Operators

Interview-Ready Definition

ANY returns TRUE if the comparison is true for AT LEAST ONE value from the subquery. ALL returns TRUE if the comparison is true for ALL values from the subquery. Used with comparison operators: = ANY is equivalent to IN, > ALL is equivalent to > MAX.

ANY vs ALL Quick Reference

Expression	Equivalent	Meaning
= ANY (subquery)	IN (subquery)	Equals at least one value
<> ALL (subquery)	NOT IN (subquery)	Not equal to any value
> ANY (subquery)	> MIN(subquery)	Greater than smallest value
> ALL (subquery)	> MAX(subquery)	Greater than largest value
< ANY (subquery)	< MAX(subquery)	Less than largest value
< ALL (subquery)	< MIN(subquery)	Less than smallest value

Example 1: ANY — Greater Than at Least One Beginner Price

▶ Beginner course prices (subquery result)

Beginner prices
699.00
999.00
1499.00

```
-- Courses priced higher than AT LEAST ONE Beginner course
-- > ANY (699, 999, 1499) means: price > 699 (the minimum)
SELECT title, price
FROM courses
WHERE price > ANY (
    SELECT price FROM courses WHERE level = 'Beginner'
)
ORDER BY price;
```

Output:

```
title                      | price    ← All courses priced > 699 qualify
---------------------------+---------
UI/UX Design Fund.         | 999.00   (> 699, YES)
MySQL Mastery              | 1299.00
Python for Data Science    | 1499.00
Business Analytics         | 1799.00
React JS Complete Guide    | 1999.00
Advanced React Patterns    | 2299.00
Flutter Mobile Dev         | 2499.00
Machine Learning A-Z       | 2999.00
Deep Learning              | 3499.00
```

Example 2: ALL — Greater Than Every Beginner Price

```
-- Courses priced higher than ALL Beginner courses
-- > ALL (699, 999, 1499) means: price > 1499 (the maximum)
SELECT title, price
FROM courses
WHERE price > ALL (
    SELECT price FROM courses WHERE level = 'Beginner'
)
ORDER BY price;

-- Equivalent (cleaner):
SELECT title, price FROM courses
WHERE price > (SELECT MAX(price) FROM courses WHERE level = 'Beginner')
ORDER BY price;
```

Output:

```
title                    | price    ← Only courses > 1499 qualify
-------------------------+---------
Business Analytics       | 1799.00
React JS Complete Guide  | 1999.00
Advanced React Patterns  | 2299.00
Flutter Mobile Dev       | 2499.00
Machine Learning A-Z     | 2999.00
Deep Learning            | 3499.00
```

How it works: > ANY = greater than the minimum value. > ALL = greater than the maximum value. The subquery returns (699, 999, 1499). ANY: need to beat just 699. ALL: need to beat all three — must be > 1499.

32. Constraints — Full Deep Dive

Interview-Ready Definition

Constraints are rules enforced by the MySQL engine to ensure data quality. They run BEFORE every INSERT, UPDATE, and DELETE. If a constraint is violated, the statement is rejected with an error. Types: PRIMARY KEY, FOREIGN KEY, UNIQUE, NOT NULL, CHECK, DEFAULT.

Example 1: NOT NULL — Mandatory Column

students table — full_name is NOT NULL

student_id	full_name	email
1	Amit Joshi	amit@gmail.com
2	Meera Nair	meera@gmail.com

```
-- This FAILS — full_name cannot be NULL:
INSERT INTO students (full_name, email, city, joined_date)
VALUES (NULL, 'newstudent@gmail.com', 'Delhi', CURDATE());
-- ERROR 1048: Column 'full_name' cannot be null

-- This ALSO FAILS — full_name was not provided:
INSERT INTO students (email, city, joined_date)
VALUES ('newstudent@gmail.com', 'Delhi', CURDATE());
-- ERROR 1364: Field 'full_name' doesn't have a default value

-- This WORKS:
INSERT INTO students (full_name, email, city, joined_date)
VALUES ('Kiran Rao', 'kiran@gmail.com', 'Delhi', CURDATE());
```

Output:

```
Query OK, 1 row affected
```

After — Kiran Rao added

student_id	full_name	email
3	Kiran Rao	kiran@gmail.com

Example 2: UNIQUE — No Duplicate Emails

```
-- students.email has UNIQUE constraint

-- First insert succeeds:
INSERT INTO students (full_name, email, city, joined_date)
VALUES ('Test User 1', 'test@gmail.com', 'Delhi', CURDATE());

-- Second insert with SAME email FAILS:
INSERT INTO students (full_name, email, city, joined_date)
VALUES ('Test User 2', 'test@gmail.com', 'Mumbai', CURDATE());
-- ERROR 1062: Duplicate entry 'test@gmail.com' for key 'students.email'

-- UNIQUE allows one NULL (NULL is "unknown" — can't be a duplicate)
INSERT INTO students (full_name, email, city, joined_date)
```

```
VALUES ('No Email User', NULL, 'Chennai', CURDATE());  -- OK! (if email is nullable)
```

Example 3: CHECK — Enforce Value Rules

```
-- CREATE TABLE with CHECK constraints
CREATE TABLE products (
  product_id  INT PRIMARY KEY AUTO_INCREMENT,
  name        VARCHAR(100) NOT NULL,
  price       DECIMAL(8,2) NOT NULL,
  stock       INT DEFAULT 0,
  discount_pct DECIMAL(5,2) DEFAULT 0,
  CONSTRAINT chk_price     CHECK (price > 0),
  CONSTRAINT chk_stock     CHECK (stock >= 0),
  CONSTRAINT chk_discount  CHECK (discount_pct BETWEEN 0 AND 100)
);

-- ✅Works:
INSERT INTO products (name, price, stock) VALUES ('Laptop', 45000.00, 10);

-- ❌Fails: price must be > 0
INSERT INTO products (name, price, stock) VALUES ('Free Item', 0.00, 5);
-- ERROR 3819: Check constraint 'chk_price' is violated.

-- ❌Fails: discount cannot be 150%
INSERT INTO products (name, price, discount_pct) VALUES ('Item', 100, 150);
-- ERROR 3819: Check constraint 'chk_discount' is violated.
```

⚠️ **Pitfall:** CHECK constraints only work in MySQL 8.0.16+. In earlier versions they are PARSED but silently IGNORED — run SELECT VERSION() to check.

Example 4: DEFAULT — Auto-Fill Values

```
CREATE TABLE courses (
  course_id  INT PRIMARY KEY AUTO_INCREMENT,
  title      VARCHAR(200) NOT NULL,
  price      DECIMAL(8,2) NOT NULL DEFAULT 0.00,       -- Default 0 if not provided
  is_active  TINYINT(1) DEFAULT 1,                     -- Default active
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP,       -- Auto timestamp
  updated_at DATETIME DEFAULT CURRENT_TIMESTAMP
             ON UPDATE CURRENT_TIMESTAMP               -- Auto-update on change
);

-- INSERT without optional columns — defaults kick in:
INSERT INTO courses (title, price) VALUES ('SQL Basics', 999.00);
-- is_active = 1 (default)
-- created_at = current timestamp
-- updated_at = current timestamp
```

📤 **Output:**

```
course_id: (auto), title: SQL Basics, price: 999.00, is_active: 1, created_at: NOW(), updated_at: NOW()
```

33. ALTER TABLE — Full Reference

Interview-Ready Definition

ALTER TABLE modifies the structure of an existing table without dropping it. Can add/remove/rename/modify columns, add/drop indexes, add/drop constraints, rename the table. WARNING: ALTER TABLE on large tables locks the table and can cause downtime.

Example 1: ADD COLUMN

students table — Before (no bio column)

student_id	full_name	email	city
1	Amit Joshi	amit@gmail.com	Hyderabad

```
-- Add a new column at the end (default position)
ALTER TABLE students ADD COLUMN bio TEXT;

-- Add at a specific position (AFTER a column)
ALTER TABLE students ADD COLUMN age INT AFTER full_name;

-- Add at the beginning
ALTER TABLE students ADD COLUMN priority_flag TINYINT(1) DEFAULT 0 FIRST;
```

Output:

```
Query OK, 0 rows affected (existing rows get NULL or DEFAULT for new column)
```

After — bio and age columns added

student_id	full_name	age	email	city	bio
1	Amit Joshi	NULL	amit@gmail.com	Hyderabad	NULL

Example 2: MODIFY COLUMN, RENAME COLUMN, DROP COLUMN

```
-- Change data type (MODIFY COLUMN)
ALTER TABLE students MODIFY COLUMN phone VARCHAR(15) NOT NULL;

-- Rename a column (MySQL 8.0+)
ALTER TABLE students RENAME COLUMN age TO student_age;

-- Change name AND type simultaneously (CHANGE COLUMN — all versions)
ALTER TABLE students CHANGE COLUMN bio biography MEDIUMTEXT;

-- Remove a column permanently (IRREVERSIBLE — all data in column is lost!)
ALTER TABLE students DROP COLUMN student_age;
ALTER TABLE students DROP COLUMN priority_flag;
```

Output:

```
Query OK for each statement
```

Pitfall: DROP COLUMN permanently deletes all data in that column — there is no undo. Always backup before dropping columns in production.

Example 3: ADD/DROP Indexes and Constraints

```
-- Add an index (speeds up searches on city)
ALTER TABLE students ADD INDEX idx_city (city);

-- Add a composite index
ALTER TABLE courses ADD INDEX idx_level_price (level, price);

-- Add a UNIQUE constraint
ALTER TABLE students ADD CONSTRAINT uq_phone UNIQUE (phone);

-- Drop an index
ALTER TABLE students DROP INDEX idx_city;

-- Drop a FOREIGN KEY constraint (need its name first)
SHOW CREATE TABLE courses;  -- find FK name, e.g., 'courses_ibfk_1'
ALTER TABLE courses DROP FOREIGN KEY courses_ibfk_1;

-- Add FOREIGN KEY back
ALTER TABLE courses ADD CONSTRAINT fk_category
  FOREIGN KEY (category_id) REFERENCES categories(category_id);

-- Rename the entire table
ALTER TABLE students RENAME TO platform_students;
-- Rename back:
ALTER TABLE platform_students RENAME TO students;
```

34. DROP TABLE & TRUNCATE TABLE

Interview-Ready Definition

DROP TABLE permanently removes the table structure AND all data. TRUNCATE TABLE removes all rows but keeps the structure — it also resets AUTO_INCREMENT. Both are DDL commands and cannot be rolled back.

Example 1: DROP TABLE

```
-- Drop a table (fails if it doesn't exist)
DROP TABLE archived_courses;

-- Safe version (no error if table doesn't exist)
DROP TABLE IF EXISTS archived_courses;

-- Drop multiple tables at once
DROP TABLE IF EXISTS temp_table1, temp_table2, temp_table3;

-- Cannot drop a parent table if children reference it:
DROP TABLE categories;
-- ERROR 1217: Cannot delete or update a parent row:
--             a foreign key constraint fails (courses references categories)

-- Must drop child first, then parent:
DROP TABLE IF EXISTS orders;
DROP TABLE IF EXISTS reviews;
DROP TABLE IF EXISTS courses;
DROP TABLE IF EXISTS students;
DROP TABLE IF EXISTS instructors;
DROP TABLE IF EXISTS categories;
```

Example 2: TRUNCATE TABLE

```
-- Remove all rows, reset AUTO_INCREMENT, keep structure
TRUNCATE TABLE temp_logs;

-- Verify: structure preserved, data gone
DESCRIBE temp_logs;          -- still shows all columns
SELECT COUNT(*) FROM temp_logs;  -- 0

-- Cannot TRUNCATE if other tables reference it via FK
TRUNCATE TABLE students;
-- ERROR: Cannot truncate a table referenced in a foreign key constraint
-- Must disable FK checks first:
SET FOREIGN_KEY_CHECKS = 0;
TRUNCATE TABLE students;
SET FOREIGN_KEY_CHECKS = 1;
```

Pitfall: SET FOREIGN_KEY_CHECKS = 0 disables all FK enforcement! Re-enable immediately after. If you forget, you can insert orphaned rows that corrupt referential integrity.

35. AUTO_INCREMENT

Interview-Ready Definition

AUTO_INCREMENT automatically generates a unique integer for each new row. It starts at 1 by default and increments by 1. The counter NEVER goes backward automatically. DELETE does not reset it; TRUNCATE does.

Example 1: AUTO_INCREMENT Behavior

students table — before inserts

student_id	full_name
(empty)	

```
INSERT INTO students (full_name, email, joined_date) VALUES ('Alice', 'a@gmail.com',
CURDATE());
INSERT INTO students (full_name, email, joined_date) VALUES ('Bob',   'b@gmail.com',
CURDATE());
INSERT INTO students (full_name, email, joined_date) VALUES ('Carol', 'c@gmail.com',
CURDATE());

DELETE FROM students WHERE full_name = 'Bob';  -- Delete row with ID=2

INSERT INTO students (full_name, email, joined_date) VALUES ('Dave', 'd@gmail.com',
CURDATE());
```

Output:

```
1 row → ID=1
2nd row → ID=2
3rd row → ID=3
Delete ID=2 (no reset)
4th row → ID=4 (not 2! gap exists)
```

After all operations

student_id	full_name
1	Alice
3	Carol
4	Dave

How it works: ID=2 (Bob) was deleted but the AUTO_INCREMENT counter moved to 3 after Bob's insert and continues forward. Dave gets ID=4, not 2. Gaps in IDs are normal and expected.

Example 2: Control AUTO_INCREMENT

```
-- Set starting value (must be > any existing ID)
ALTER TABLE students AUTO_INCREMENT = 1000;
-- Next insert will get ID=1000

-- Check current AUTO_INCREMENT value
SELECT AUTO_INCREMENT
FROM information_schema.TABLES
WHERE TABLE_SCHEMA = 'edushop'
  AND TABLE_NAME   = 'students';
```

```
-- Get last inserted ID in current session
INSERT INTO students (full_name, email, joined_date) VALUES ('Eve', 'eve@gmail.com',
CURDATE());
SELECT LAST_INSERT_ID();  -- Returns 1000 (the just-inserted ID)
```

Output:

```
AUTO_INCREMENT = 1000

Next insert → student_id = 1000
LAST_INSERT_ID() = 1000
```

Pitfall: Never assume AUTO_INCREMENT IDs are sequential or gap-free. Failed transactions, deletes, and ROLLBACKS all consume IDs without creating rows. Design systems to treat IDs as unique identifiers, not sequential counters.

36. MySQL Comments

Interview-Ready Definition

SQL comments document code and are ignored by the MySQL engine. Three styles: -- (single line), # (single line MySQL-specific), /* */ (multi-line block comment). Use comments liberally to explain why, not what.

Example 1: All Comment Styles

```
-- This is a single-line comment (standard SQL)
SELECT * FROM students;   -- inline comment after statement

# This is a MySQL-specific single-line comment
SELECT * FROM courses; # another inline MySQL comment

/* This is a
   multi-line block comment.
   Great for documentation headers. */
SELECT * FROM orders;

/*
  Query: Monthly Revenue Report
  Author: Ravi Kumar
  Date: 2024-01-15
  Purpose: Calculate total and average revenue per month
*/
SELECT
  YEAR(order_date)                AS year,
  MONTH(order_date)               AS month,
  COUNT(*)                        AS orders,   -- total order count
  SUM(amount_paid)                AS revenue,  -- total revenue
  ROUND(AVG(amount_paid), 2)      AS avg_order -- average order value
FROM orders
WHERE status = 'completed'        /* only completed orders */
GROUP BY YEAR(order_date), MONTH(order_date)
ORDER BY year, month;
```

Tip: Use comments to explain business logic and WHY a query works a certain way — not just what it does. Future maintainers (including yourself 6 months later) will thank you.

37. Part 1 — Interview Questions & Answers

These are the most frequently asked MySQL interview questions for fresher and junior roles, covering all Part 1 concepts.

❓ What is MySQL and what is it used for?

✅MySQL is an open-source Relational Database Management System (RDBMS) that stores data in tables and uses SQL. Used by Facebook, Twitter, YouTube, and millions of applications for reliable structured data storage with ACID transactions, foreign key constraints, and rich query capabilities.

❓ What is the difference between DDL, DML, DQL, DCL, and TCL?

✅DDL (CREATE, ALTER, DROP, TRUNCATE) — defines structure; auto-committed. DML (INSERT, UPDATE, DELETE) — manipulates data; can be rolled back. DQL (SELECT) — queries data. DCL (GRANT, REVOKE) — controls access permissions. TCL (COMMIT, ROLLBACK, SAVEPOINT) — controls transactions.

❓ What is a PRIMARY KEY? Can it be NULL?

✅A PRIMARY KEY uniquely identifies each row. It is automatically NOT NULL and creates a unique index. A table can have only one PRIMARY KEY (can be single or composite). It CANNOT contain NULL values.

❓ What is the difference between PRIMARY KEY and UNIQUE key?

✅PRIMARY KEY: unique + NOT NULL + only one per table + is the clustered index in InnoDB. UNIQUE key: unique + allows exactly ONE NULL + multiple can exist per table + creates a non-clustered index.

❓ What is a FOREIGN KEY and what is referential integrity?

✅A FOREIGN KEY is a column that references the PRIMARY KEY of another table, establishing a parent-child relationship. Referential integrity ensures child rows cannot reference non-existent parent rows — MySQL enforces this with FK constraints.

❓ What are the referential actions for ON DELETE?

✅RESTRICT (default) — prevents parent delete if children exist. CASCADE — deletes child rows automatically. SET NULL — sets child FK to NULL. SET DEFAULT — sets to default value. NO ACTION — like RESTRICT but checked at end of transaction.

❓ What is the difference between WHERE and HAVING?

✅WHERE filters INDIVIDUAL ROWS before grouping — runs at step 2. HAVING filters GROUPS after GROUP BY — runs at step 5. WHERE cannot use aggregate functions (SUM, COUNT). HAVING can. Rule: if you have GROUP BY, think of HAVING for aggregate conditions.

❓ What is the logical execution order of a SELECT statement?

✅FROM → JOIN → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → LIMIT. This explains: aliases not usable in WHERE (created in SELECT step), aggregate functions not usable in WHERE (groups created in GROUP BY step), ORDER BY can use SELECT aliases.

❓ Explain the difference between INNER JOIN, LEFT JOIN, and RIGHT JOIN.

✅INNER JOIN: returns only matching rows from both tables. LEFT JOIN: all rows from left table + matching from right (NULLs for no match). RIGHT JOIN: all rows from right table + matching from left (NULLs for no match). LEFT and RIGHT JOIN are equivalent — just swap table order.

❓ What is a SELF JOIN? Give an example.

✅A SELF JOIN joins a table to itself using two different aliases. Example: to find courses in the same category: FROM courses a JOIN courses b ON a.category_id = b.category_id AND a.course_id < b.course_id. Used for hierarchies (employee-manager), sibling records, and duplicate detection.

❓ What is the difference between UNION and UNION ALL?

✅UNION combines result sets and removes duplicate rows — slower due to sorting for deduplication. UNION ALL combines result sets and keeps all rows including duplicates — faster. Both require same column count and compatible types. Use UNION ALL when duplicates are acceptable or known not to exist.

❓ What is NULL? How do you check for it?

✅NULL means "unknown" or "no value" — not zero, not empty string. Any comparison with NULL produces NULL. Check using IS NULL or IS NOT NULL. Never use = NULL (always evaluates to NULL, never TRUE). Use IFNULL(), COALESCE(), or NULLIF() to handle NULLs in expressions.

❓ What is the difference between DELETE, TRUNCATE, and DROP?

✅DELETE: DML — removes specific rows (with WHERE), can be rolled back, fires triggers, does not reset AUTO_INCREMENT. TRUNCATE: DDL — removes all rows, cannot be rolled back, does not fire row triggers, resets AUTO_INCREMENT. DROP: DDL — removes the entire table structure and all data.

❓ What is AUTO_INCREMENT? What happens when you delete rows?

✅AUTO_INCREMENT generates unique sequential integers for new rows. When you DELETE rows, the counter does NOT reset — gaps appear in IDs. This is by design: IDs are unique identifiers, not sequential row numbers. TRUNCATE resets the counter; DELETE does not.

❓ What is BETWEEN and is it inclusive?

✅BETWEEN low AND high tests if a value falls within a range. It is INCLUSIVE — equivalent to col >= low AND col <= high. For DATETIME columns, always include 23:59:59 in the upper bound or use < next_day to avoid missing records at the end of the day.

❓ Why is NOT IN dangerous with NULLs in the subquery?

✅If the subquery returns any NULL values, NOT IN returns no rows at all — because col NOT IN (..., NULL) evaluates to col != NULL which is always NULL (not TRUE). Always prefer NOT EXISTS or LEFT JOIN + IS NULL instead of NOT IN with subqueries.

❓ What is the difference between CHAR and VARCHAR?

✅CHAR(n) always stores exactly n bytes — pads with spaces. VARCHAR(n) stores actual length + 1 or 2 bytes overhead. CHAR is faster for fixed-length data (country codes: IN, US). VARCHAR is more storage-efficient for variable-length data (names, emails).

❓ Why use DECIMAL instead of FLOAT for money?

✅FLOAT uses binary fractions internally — 0.1 + 0.2 can equal 0.30000000000000004. DECIMAL stores exact decimal digits. For financial calculations, always use DECIMAL(10,2). A rounding error of 0.01 per transaction × millions of transactions = significant financial discrepancy.

❓ What is a composite key?

✅A composite (compound) key uses two or more columns together as the unique identifier. Example: PRIMARY KEY (student_id, course_id) in an enrollment table — neither alone is unique, but the combination is. Also useful for composite UNIQUE constraints and composite indexes.

❓ What is the difference between DATETIME and TIMESTAMP?

✅DATETIME stores the exact date-time with no timezone conversion — what you insert is what you get. TIMESTAMP converts to UTC on storage and back to session timezone on retrieval. TIMESTAMP range: 1970-2038. Use DATETIME for historical records; TIMESTAMP for auto-updated created_at/updated_at.

End of Part 1 — Coming in Part 2

Part 2 Topics

Part 2 covers the advanced MySQL concepts. Here is what is coming:

- String Functions — CONCAT, SUBSTRING, REPLACE, TRIM, LPAD, REGEXP, FORMAT (20+ functions with examples)
- Numeric Functions — ROUND, FLOOR, CEIL, ABS, MOD, POWER, SQRT, FORMAT
- Date & Time Functions — NOW, CURDATE, DATE_FORMAT, DATEDIFF, DATE_ADD, YEAR/MONTH/DAY, TIMESTAMPDIFF (20+ functions)
- Subqueries — Scalar, Multi-row, Correlated, Derived Tables, EXISTS
- Window Functions — ROW_NUMBER, RANK, DENSE_RANK, NTILE, LAG, LEAD, FIRST_VALUE, LAST_VALUE, NTH_VALUE, Running Totals
- Common Table Expressions (CTEs) — Simple, Multiple, Recursive CTEs
- Views — CREATE VIEW, Updatable Views, WITH CHECK OPTION
- Triggers — BEFORE/AFTER INSERT/UPDATE/DELETE, SIGNAL for custom errors
- Stored Procedures — IN/OUT/INOUT params, IF-ELSE, CASE, LOOP, CURSOR
- Stored Functions — User-defined scalar functions
- Transactions & ACID — START TRANSACTION, COMMIT, ROLLBACK, SAVEPOINT, Isolation Levels
- Indexes — B-Tree, FULLTEXT, Composite, EXPLAIN analysis, best practices
- Normalization — 1NF, 2NF, 3NF, BCNF with EduShop examples
- Error Handling — SIGNAL SQLSTATE, DECLARE HANDLER
- Performance Optimization — 10 essential tips with before/after comparisons
- Section A: 30 Advanced Interview Questions with detailed answers
- Section B: 15 Coding Challenges with multiple solution approaches + which is optimal